

EVALUASI TENGAH SEMESTER TEKNOLOGI IOT

Dosen:

Ahmad Radhy, S.Si., M.Si.

Smart Mushroom House: IoT Monitoring Suhu dan Kelembaban Berbasis Rust dan ESP32-S3



Disusun Oleh:

Muhammad Naufal Zuhair

2042231005

Kania Nayaka Utamaysarah

20422310033

**PROGRAM STUDI D4 TEKNOLOGI REKAYASA
INSTRUMENTASI
DEPARTEMEN TEKNIK INSTRUMENTASI
FAKULTAS VOKASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2025**

DAFTAR ISI

1	PENDAHULUAN	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	2
1.3	Tujuan	2
1.4	Manfaat	3
1.5	Batasan Masalah	3
2	TINJAUAN PUSTAKA	4
2.1	Internet of Things (IoT)	4
2.2	Mikrokontroler ESP32-S3	5
2.3	Bahasa Pemrograman Rust untuk Embedded System	6
2.4	Sensor DHT22	7
2.5	Protokol MQTT	8
2.6	Platform ThingsBoard Cloud	9
2.7	Over The Air (OTA)	9
2.8	Firmware pada Sistem IoT	10
2.9	State of the Art	11
2.10	Kumbung Jamur	16
3	METODOLOGI	17
3.1	Metodologi Penelitian	17
3.2	Desain Arsitektur Sistem	18
3.3	Desain Perangkat Keras	19
3.4	Desain Perangkat Lunak	20
3.5	Alur Data dan Konektivitas MQTT–ThingsBoard–OTA	21
3.6	Mekanisme OTA (Over-The-Air Update)	21
3.7	Rangka Alur Proses Sistem	22
4	IMPLEMENTASI & PENGUJIAN SISTEM	23
4.1	Implementasi Perangkat Lunak	23
4.2	Struktur Program Muhsroom House	23
4.3	Implementasi Koneksi Wi-Fi dan MQTT	24
4.4	Pembacaan Sensor DHT22	25
4.5	Implementasi OTA (Over-The-Air) Update	25

4.6	Konfigurasi ThingsBoard Cloud	26
4.7	Pengujian Sistem	26
4.8	Analisis Latensi dan Performa Sistem	27
4.9	Analisis Keamanan Sistem	27
5	HASIL DAN ANALISIS	28
5.1	Hasil Implementasi Sistem	28
5.2	Analisis Telemetry dan Latensi dengan Gnuplot	28
5.3	Analisis Keamanan dan Keandalan Sistem	29
6	KESIMPULAN DAN SARAN	30
6.1	Kesimpulan	30
6.2	Saran	30
A	LAMPIRAN	32

DAFTAR GAMBAR

2.1	Internet of Things	4
2.2	ESP32-S3	5
2.3	Bahasa Rust	6
2.4	Sensor DHT22	7
2.5	Protokol MQTT	8
2.6	Thingsboard Cloud	9
2.7	Over The Air (OTA)	10
2.8	Firmware	11
2.9	Kumbung Jamur	16
3.1	Arsitektur Sistem	18
3.2	Desain Perangkat Keras	19
A.1	Streaming Data Terminal dan Thingsboards	32
A.2	Tampilan Grafik Temperature dan Humidity di GNUPLOT	33
A.3	Tampilan Grafik Perbandingan Latensi Thingsboard dan ESP32 di GNU-PLOT	33
A.4	Bukti Update OTA	34

Bab 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah membawa perubahan besar dalam berbagai bidang kehidupan, termasuk sektor pertanian. Salah satu inovasi yang berperan penting dalam era modern ini adalah Internet of Things (IoT), yaitu konsep yang memungkinkan berbagai perangkat fisik seperti sensor, aktuator, dan mikrokontroler saling terhubung melalui jaringan internet untuk mengumpulkan, mengirim, serta memproses data secara otomatis. Teknologi ini membuka peluang besar dalam penerapan pertanian cerdas (smart farming), di mana proses pemantauan dan pengendalian lingkungan dapat dilakukan secara efisien dan real-time. Salah satu penerapannya adalah pada sistem pemantauan kumbung jamur, yang bertujuan menjaga kondisi lingkungan tetap ideal bagi pertumbuhan jamur.

Dalam proses budidaya jamur, suhu dan kelembaban merupakan dua parameter utama yang harus dijaga agar jamur dapat tumbuh optimal. Kelembaban yang terlalu rendah dapat menghambat pembentukan tubuh buah, sedangkan suhu yang terlalu tinggi dapat menyebabkan jamur cepat layu atau mati. Oleh karena itu, dibutuhkan sistem yang mampu memantau kondisi lingkungan kumbung secara otomatis, real-time, dan dapat diakses dari jarak jauh untuk menjaga kestabilan kondisi tersebut.

Proyek Mushroom House dikembangkan untuk menjawab kebutuhan tersebut dengan merancang sistem pemantauan suhu dan kelembaban berbasis mikrokontroler ESP32-S3 dan sensor DHT22, yang dikendalikan menggunakan bahasa pemrograman Rust. Bahasa Rust dipilih karena memiliki keunggulan dalam hal keamanan memori (memory safety), efisiensi tinggi, serta performa optimal tanpa bergantung pada garbage collector seperti Python atau Java. Selain itu, Rust juga mendukung pemrograman konkuren (concurrent programming) secara aman, sehingga dapat meningkatkan kecepatan dan stabilitas sistem tertanam (embedded system).

Sebagai fitur tambahan, Mushroom House dilengkapi dengan mekanisme Over-The-Air (OTA), yang memungkinkan pembaruan firmware dilakukan secara jarak jauh melalui jaringan internet tanpa perlu melakukan flashing manual. Fitur ini menjadi nilai tambah penting dalam pengelolaan perangkat IoT berskala besar karena mempermudah proses pemeliharaan, meningkatkan keamanan, dan mendukung pengembangan sistem

secara berkelanjutan. Dengan memanfaatkan kombinasi antara Rust, ESP32-S3, protokol MQTT, dan platform ThingsBoard, proyek ini diharapkan dapat menghadirkan solusi IoT yang andal, efisien, dan mudah dikembangkan untuk sistem pemantauan lingkungan pada kumbung jamur.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan sistem pemantauan suhu dan kelembaban pada kumbung jamur menggunakan mikrokontroler ESP32-S3 dan sensor DHT22?
2. Bagaimana cara mengirimkan data hasil pembacaan sensor ke platform ThingsBoard Cloud melalui protokol MQTT agar dapat dimonitor secara real-time?
3. Bagaimana penerapan bahasa Rust dapat meningkatkan efisiensi, keamanan memori, dan stabilitas sistem dibandingkan bahasa pemrograman lain yang umum digunakan pada perangkat IoT?
4. Bagaimana mekanisme Over-The-Air (OTA) dapat diterapkan untuk memperbarui firmware sistem tanpa perlu melakukan flashing manual?

1.3 Tujuan

Tujuan dari pengembangan proyek Mushroom House ini adalah:

- Merancang dan mengimplementasikan sistem pemantauan suhu dan kelembaban pada kumbung jamur berbasis IoT.
- Mengintegrasikan mikrokontroler ESP32-S3, sensor DHT22, dan platform ThingsBoard Cloud menggunakan protokol komunikasi MQTT.
- Mengembangkan perangkat lunak sistem menggunakan bahasa Rust untuk mendapatkan performa tinggi dan keamanan memori yang baik.
- Mengimplementasikan fitur OTA (Over-The-Air) agar proses pembaruan firmware dapat dilakukan secara jarak jauh.
- Menyediakan antarmuka pemantauan berbasis web Thingsboard yang memudahkan pengguna untuk memantau kondisi lingkungan kumbung secara real-time.

1.4 Manfaat

Proyek ini diharapkan dapat memberikan beberapa manfaat sebagai berikut:

- **Bagi petani jamur**, sistem ini dapat membantu memantau kondisi suhu dan kelembaban kumbung secara lebih praktis dan efisien tanpa harus melakukan pengecekan langsung setiap saat. Dengan demikian, proses budidaya jamur dapat lebih terkontrol dan hasil panen bisa lebih optimal.
- **Bagi kalangan akademisi**, proyek ini dapat menjadi contoh penerapan teknologi Internet of Things (IoT) dalam bidang pertanian, khususnya pada budidaya jamur, serta sebagai bahan pembelajaran mengenai penggunaan bahasa Rust pada sistem tertanam.
- **Bagi pengembang atau mahasiswa teknik**, proyek ini dapat menjadi referensi dalam pengembangan sistem IoT yang aman dan efisien, terutama yang melibatkan komunikasi MQTT dan fitur pembaruan firmware jarak jauh (OTA).
- **Bagi dunia industri dan penelitian**, sistem ini dapat menjadi dasar pengembangan lebih lanjut menuju pertanian cerdas (smart farming) yang mampu memantau dan mengendalikan kondisi lingkungan secara otomatis dengan biaya yang relatif terjangkau.

1.5 Batasan Masalah

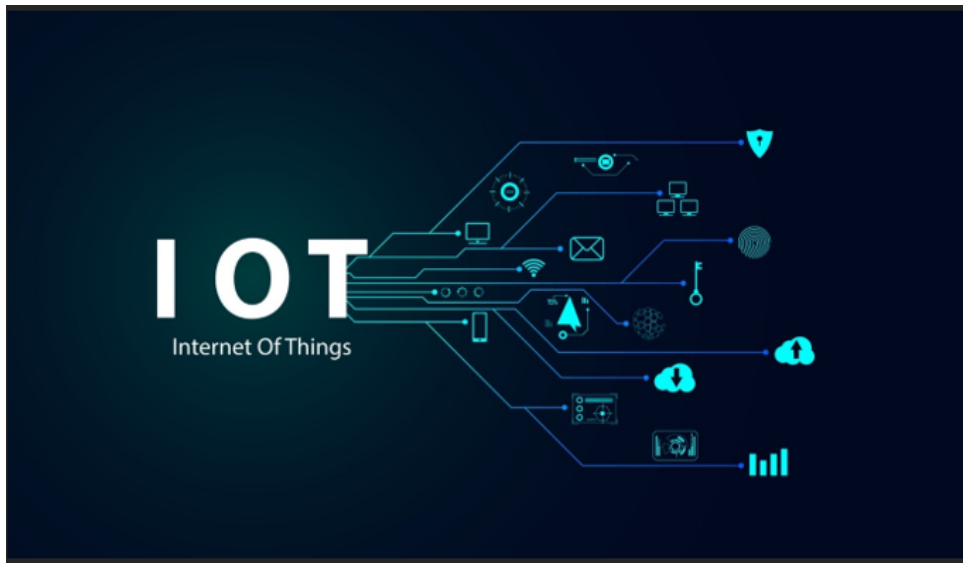
Agar pembahasan lebih terfokus, proyek ini dibatasi oleh hal-hal berikut:

- Sistem hanya melakukan pemantauan suhu dan kelembaban, tanpa fitur pengendalian otomatis (seperti kipas, mist maker, atau pemanas).
- Sensor yang digunakan adalah DHT22, dengan satu titik pengukuran di dalam kumbung jamur.
- Platform pemantauan yang digunakan adalah ThingsBoard Cloud, dengan komunikasi melalui protokol MQTT.
- Mikrokontroler yang digunakan adalah ESP32-S3, dan seluruh perangkat lunak dikembangkan menggunakan bahasa pemrograman Rust.
- Sistem diuji dalam skala prototipe, bukan pada kumbung jamur komersial berskala besar.
- Koneksi jaringan menggunakan Wi-Fi, sehingga performa sistem bergantung pada kestabilan jaringan internet.

Bab 2

TINJAUAN PUSTAKA

2.1 Internet of Things (IoT)



Gambar 2.1: Internet of Things

Internet of Things (IoT) adalah konsep jaringan yang menghubungkan berbagai perangkat agar dapat saling berkomunikasi melalui internet untuk bertukar data dan berinteraksi antara dunia fisik dan digital. Secara umum, IoT terdiri dari tiga komponen utama, yaitu sensor atau aktuator, jaringan komunikasi, dan platform aplikasi. Sensor berfungsi mendeteksi dan mengubah besaran fisik menjadi data digital, kemudian jaringan komunikasi mengirimkan data tersebut ke server, sedangkan platform aplikasi menampilkan hasilnya dalam bentuk informasi yang mudah dipahami pengguna.

Dalam penerapannya, sistem IoT modern biasanya menggunakan protokol komunikasi ringan seperti MQTT atau CoAP karena sebagian besar perangkat IoT memiliki keterbatasan daya dan kapasitas pemrosesan. Di antara berbagai protokol yang ada, MQTT menjadi salah satu yang paling banyak digunakan karena mengusung arsitektur publish-subscribe yang mendukung komunikasi asynchronous serta efisien dalam penggunaan bandwidth.

Teknologi IoT kini tidak hanya digunakan di bidang industri atau transportasi, tetapi juga berkembang pesat pada sektor rumah pintar (smart home). Melalui integrasi sen-

sor dan aktuator, pengguna dapat memantau serta mengendalikan sistem pencahayaan, suhu ruangan, keamanan, dan berbagai peralatan listrik secara jarak jauh melalui jaringan internet. Salah satu contoh penerapan konsep ini adalah Mushroom House, sebuah sistem rumah pintar yang memanfaatkan komunikasi MQTT dan integrasi cloud untuk menghadirkan pemantauan dan pengendalian perangkat secara efisien dan terhubung.

2.2 Mikrokontroler ESP32-S3



Gambar 2.2: ESP32-S3

ESP32-S3 merupakan mikrokontroler System-on-Chip (SoC) generasi terbaru keluaran Espressif yang dikembangkan khusus untuk kebutuhan Internet of Things (IoT) dan edge computing. Perangkat ini dibekali dengan prosesor Xtensa LX7 dual-core berkecepatan hingga 240 MHz, serta mendukung vector instruction yang dapat mempercepat proses komputasi kecerdasan buatan (AI acceleration).

Dibandingkan dengan pendahulunya, yaitu ESP32 dan ESP32-S2, seri ESP32-S3 menawarkan peningkatan pada sisi kinerja prosesor, efisiensi konsumsi daya, serta dukungan fitur keamanan seperti secure boot dan flash encryption yang melindungi integritas firmware. Selain itu, mikrokontroler ini dilengkapi dengan berbagai antarmuka komunikasi seperti UART, SPI, I2C, I2S, dan PWM, yang memudahkan integrasi dengan beragam sensor maupun aktuator.

Modul Wi-Fi 2.4 GHz yang sudah terpasang di dalamnya memungkinkan perangkat terhubung langsung ke internet tanpa memerlukan modul tambahan. Dalam proyek Mushroom House, ESP32-S3 berperan sebagai unit akuisisi data (data acquisition unit) yang membaca informasi dari sensor DHT22, memproses data tersebut, dan mengirimkannya ke ThingsBoard Cloud melalui protokol MQTT.

Dukungan terhadap ESP-IDF SDK yang matang menjadikan ESP32-S3 fleksibel untuk diprogram menggunakan berbagai bahasa seperti C/C++, MicroPython, maupun Rust. Melalui pustaka `esp-idf-svc` dan `embedded-svc`, bahasa Rust dapat langsung memanfaatkan layanan sistem seperti Wi-Fi, MQTT, HTTP, serta Over-The-Air (OTA). Dengan kemampuan tersebut, ESP32-S3 menjadi platform yang ideal untuk pengembangan dan eksperimen sistem IoT berbasis Rust yang aman, efisien, dan andal.

2.3 Bahasa Pemrograman Rust untuk Embedded System



Gambar 2.3: Bahasa Rust

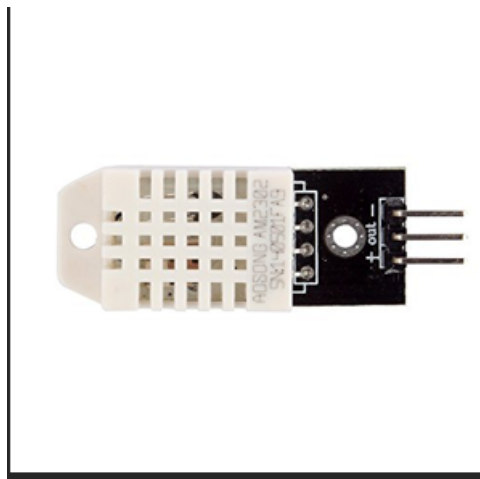
Rust merupakan bahasa pemrograman yang dikembangkan oleh Mozilla dengan tujuan untuk menjadi alternatif bagi C/C++ dalam pengembangan sistem tingkat rendah (low-level system programming) yang membutuhkan efisiensi tinggi sekaligus keamanan memori yang ketat. Rust memperkenalkan konsep *ownership* dan *borrowing*, di mana setiap data hanya memiliki satu pemilik pada satu waktu. Mekanisme ini secara efektif mencegah terjadinya *dangling pointer*, *data race*, maupun *memory leak* yang sering muncul pada bahasa pemrograman konvensional seperti C.

Keunggulan utama Rust terletak pada kemampuannya memberikan performa setara C, namun tetap menjamin keamanan dan stabilitas sistem. Dalam konteks sistem tertanam (embedded system), hal ini menjadi sangat penting karena perangkat seperti ESP32-S3 memiliki sumber daya memori terbatas dan tidak toleran terhadap kesalahan pengelolaan memori.

Selain itu, Rust juga mendukung *asynchronous programming*, yang memungkinkan beberapa proses berjalan secara bersamaan tanpa saling mengganggu, sehingga meningkatkan efisiensi dan responsivitas sistem. Pada proyek Mushroom house, bahasa Rust digunakan untuk mengembangkan firmware yang menangani berbagai fungsi utama, seperti pengelolaan koneksi Wi-Fi, komunikasi MQTT, pembacaan data dari sensor DHT22, serta pelaksanaan pembaruan firmware OTA (Over-The-Air).

Proses modularisasi kode dilakukan melalui sistem Cargo crate, yang mempermudah manajemen dependensi dan struktur proyek. Keberhasilan penerapan Rust pada platform ESP32-S3 menunjukkan bahwa bahasa ini sudah cukup matang untuk digunakan dalam pengembangan sistem IoT tertanam yang membutuhkan kombinasi antara keamanan, efisiensi, dan performa tinggi.

2.4 Sensor DHT22



Gambar 2.4: Sensor DHT22

Sensor DHT22 atau dikenal juga dengan AM2302 merupakan sensor digital yang berfungsi untuk mengukur suhu dan kelembaban udara dengan tingkat akurasi yang cukup tinggi. Sensor ini bekerja berdasarkan dua prinsip utama, yaitu *capacitive humidity sensing* untuk mendeteksi kelembaban relatif (Relative Humidity/RH) dan *thermistor* untuk mengukur suhu udara di sekitarnya. Hasil pembacaan dari DHT22 dikirimkan dalam bentuk data digital 40 bit, yang terdiri atas 16 bit untuk nilai kelembaban, 16 bit untuk suhu, serta 8 bit tambahan sebagai checksum untuk memastikan validitas data.

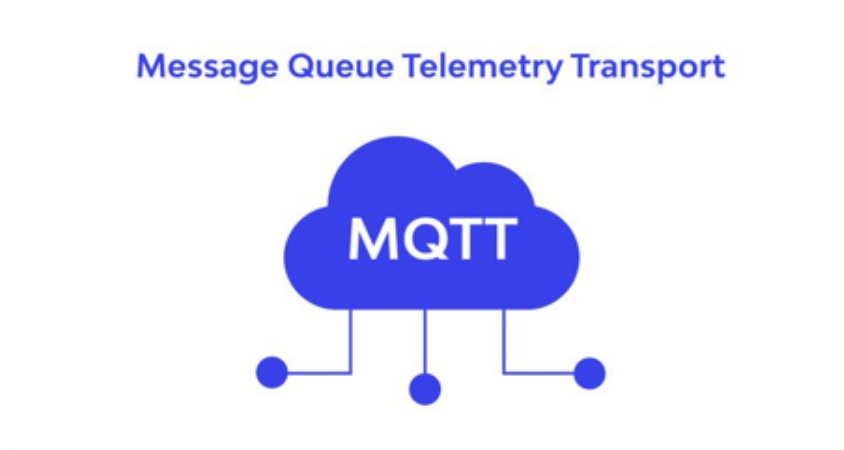
Dibandingkan dengan DHT11, sensor DHT22 memiliki cakupan pengukuran yang lebih luas, akurasi yang lebih baik, dan kemampuan bekerja pada rentang suhu ekstrem dari -40°C hingga 80°C . Selain itu, sensor ini sudah dikalibrasi dari pabrik dan memiliki stabilitas jangka panjang yang baik, sehingga sangat cocok digunakan pada sistem pemantauan lingkungan.

Dalam sistem Mushroom house, DHT22 dihubungkan ke pin GPIO pada ESP32-S3 melalui komunikasi single-wire digital, yang memungkinkan transfer data sederhana dan efisien. Sensor ini memiliki interval pembaruan data maksimal dua detik per siklus, menjadikannya ideal untuk aplikasi pemantauan seperti kumbung jamur, yang tidak memerlukan pembaruan data berkecepatan tinggi.

Data yang diperoleh dari DHT22 kemudian diproses dan dikonversi ke format JSON,

sebelum dikirim ke ThingsBoard Cloud melalui protokol MQTT untuk ditampilkan secara real-time. Pemilihan sensor DHT22 pada proyek ini didasarkan pada kombinasi antara akurasi tinggi, konsumsi daya yang rendah, serta kemudahan integrasi dengan sistem berbasis mikrokontroler seperti ESP32-S3.

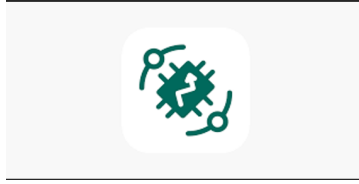
2.5 Protokol MQTT



Gambar 2.5: Protokol MQTT

Protokol MQTT (Message Queuing Telemetry Transport) merupakan salah satu protokol komunikasi yang banyak digunakan dalam sistem Internet of Things (IoT) karena ringan dan efisien. MQTT bekerja dengan konsep publish/subscribe, di mana suatu perangkat dapat mengirimkan data ke broker melalui topik tertentu (publish), sementara perangkat lain yang berlangganan pada topik tersebut (subscribe) akan menerima data secara otomatis. Keunggulan utama protokol ini terletak pada efisiensi penggunaan bandwidth serta kemampuannya menjaga keandalan komunikasi meskipun jaringan mengalami latensi tinggi atau koneksi tidak stabil.

Dalam sistem Muhsroom House, protokol MQTT dimanfaatkan untuk mengirimkan data suhu dan kelembaban dari sensor DHT22 ke platform ThingsBoard Cloud secara berkala. Sistem ini menggunakan Quality of Service (QoS) level 1 untuk memastikan data telemetry tetap terkirim meskipun terjadi gangguan, serta QoS level 2 untuk komunikasi pembaruan firmware agar tidak terjadi duplikasi data. Dengan penerapan MQTT, Muhsroom House mampu mempertahankan komunikasi dua arah secara efisien antara perangkat dan server, baik untuk pemantauan data secara real-time maupun untuk pengendalian jarak jauh.



Gambar 2.6: Thingsboard Cloud

2.6 Platform ThingsBoard Cloud

ThingsBoard adalah platform IoT berbasis open-source yang berfungsi sebagai pusat manajemen perangkat, pengumpulan data sensor, serta penyajian informasi melalui tampilan dashboard interaktif. Platform ini mendukung berbagai protokol komunikasi seperti MQTT, HTTP, dan CoAP, sehingga fleksibel digunakan pada beragam sistem IoT. Selain itu, ThingsBoard juga dilengkapi dengan fitur *device provisioning*, penyimpanan *telemetry* berbasis *time-series database*, serta *rule engine* yang memungkinkan otomatisasi proses sesuai kondisi tertentu.

Dalam sistem Muhsroom House, ThingsBoard berperan sebagai pusat pengelolaan data dan pembaruan firmware jarak jauh (Over-The-Air atau OTA). Data suhu dan kelembaban yang dikirim oleh modul ESP32-S3 disimpan di basis data time-series dan divisualisasikan dalam bentuk grafik dan indikator yang mudah dipahami. Melalui dashboard, pengguna dapat memantau kondisi rumah secara real-time, memeriksa status perangkat, serta melakukan pembaruan firmware melalui perintah Remote Procedure Call (RPC). Keunggulan lain dari ThingsBoard adalah kemampuannya untuk dijalankan baik di cloud maupun server lokal, dukungan terhadap enkripsi TLS/SSL untuk keamanan data, serta kompatibilitas dengan sistem analitik seperti Kafka dan Grafana untuk pengolahan data lanjutan.

2.7 Over The Air (OTA)

Over-The-Air (OTA) update merupakan mekanisme pembaruan firmware atau perangkat lunak secara jarak jauh yang memungkinkan perangkat IoT memperoleh versi terbaru tanpa perlu intervensi langsung. Teknologi ini menjadi elemen penting dalam sistem IoT modern, terutama karena banyak perangkat tersebar di lokasi yang sulit dijangkau atau dalam jumlah besar. OTA biasanya memanfaatkan protokol komunikasi ringan seperti MQTT, HTTP, atau CoAP untuk mentransfer berkas firmware dari server ke perangkat. Setelah proses unduhan selesai, sistem akan melakukan verifikasi integritas melalui checksum atau tanda tangan digital sebelum firmware baru diaktifkan. Umumnya, arsitektur OTA terdiri atas tiga komponen utama: server firmware, broker komunikasi, dan klien IoT yang mengatur proses unduh dan aktivasi. Dengan mekanisme ini, OTA mendukung



Gambar 2.7: Over The Air (OTA)

pemeliharaan sistem secara efisien, memperpanjang umur operasional perangkat, serta menjaga keamanan dari celah yang telah diperbaiki.

Dari sisi keamanan dan keandalan, sistem OTA harus memastikan bahwa proses pembaruan tidak dapat dimanipulasi serta memiliki kemampuan pemulihan jika terjadi kegagalan (fail-safe). Implementasi modern biasanya menggunakan enkripsi TLS untuk melindungi data selama transmisi, autentikasi berbasis token unik per perangkat, dan partisi ganda (primary dan rollback) agar perangkat tetap berfungsi bila pembaruan gagal.

Dalam proyek Muhsroom House, konsep OTA diimplementasikan menggunakan bahasa Rust pada ESP32-S3, dengan ThingsBoard sebagai pengendali proses pembaruan melalui perintah Remote Procedure Call (RPC) yang berisi parameter `ota_url`. File firmware diunduh melalui `EspMQTTConnection`, kemudian diverifikasi dan diaktifkan secara otomatis. Keunggulan Muhsroom House terletak pada penggunaan Rust yang memiliki sistem manajemen memori aman—bebas dari *memory corruption* dan *data race*—sehingga pembaruan dapat dijalankan secara paralel dengan reliabilitas tinggi. Pendekatan ini menunjukkan bahwa integrasi antara Rust, MQTT, dan OTA mampu menghasilkan sistem IoT yang tangguh, efisien, serta mudah dikelola pada skala besar.

2.8 Firmware pada Sistem IoT

Firmware merupakan perangkat lunak tertanam yang berperan mengendalikan operasi dasar perangkat keras seperti sensor, mikrokontroler, dan modul komunikasi pada sistem IoT. Firmware berfungsi sebagai penghubung antara perangkat keras dengan sistem jaringan, mencakup proses pembacaan sensor, pengiriman data ke server, hingga pengendalian aktuator sesuai perintah yang diterima. Karena dijalankan pada perangkat dengan sumber daya terbatas, firmware harus dirancang dengan efisiensi tinggi, stabilitas yang



Gambar 2.8: Firmware

baik, serta keamanan memori yang terjamin. Bahasa pemrograman seperti C, C++, dan Rust sering digunakan untuk menghasilkan kode biner yang ringan namun tetap andal dan aman. Dalam penerapannya, firmware modern tidak only mengatur logika dasar perangkat, tetapi juga menangani komunikasi terenkripsi, autentikasi berbasis token, serta pembaruan sistem jarak jauh melalui mekanisme Over-The-Air (OTA).

Pada proyek Muhsroom House, firmware dikembangkan menggunakan bahasa Rust pada platform ESP32-S3 dengan memanfaatkan pustaka `esp-idf-svc`. Struktur programnya mencakup proses inisialisasi Wi-Fi, koneksi ke broker MQTT milik ThingsBoard, pengambilan data dari sensor DHT22, serta pelaksanaan pembaruan otomatis melalui fitur OTA. Dengan pendekatan *asynchronous non-blocking*, sistem mampu menjalankan beberapa proses secara bersamaan tanpa mengganggu stabilitas utama perangkat. Hasilnya, firmware Muhsroom House tidak only efisien dan aman dari *data race*, tetapi juga mudah dipelihara melalui pembaruan versi secara jarak jauh, menjadikannya solusi yang handal untuk implementasi IoT berskala rumah pintar.

2.9 State of the Art

Tabel 2.1: State of the Art Penelitian Terkait

No.	Judul & Penulis	Metode yang Digunakan	Hasil & Temuan Utama	Relevansi terhadap Penelitian
1	A Web-Based Supersorbent Polymer Solver in Simple Science: PHP, Python, Fortran, and GNUPlot Together – Miguel Casquilho, 2022	Solver ilmiah berbasis web dengan GNUPlot untuk visualisasi perhitungan matematis.	Web-based solver efektif untuk analisis numerik.	Dasar integrasi GNUPlot sebagai alat analisis latensi & data sensor suhu/kelembaban.
2	Acceptance Sampling in Quality Control: From Theory to the Web – Rocha et al., 2023	Mengotomatisasi acceptance sampling melalui platform web dengan visualisasi statistik GNUPlot.	PHP–Python–GNUPlot terbukti efektif untuk analisis web.	Acuan penggunaan GNUPlot dalam analisis ilmiah sistem IoT monitoring.
3	Ariel OS: An Embedded Rust Operating System for Networked Sensors & Multi-Core Microcontrollers – Frank et al., 2025	OS tertanam berbasis Rust untuk ESP32-S3 & RISC-V; mendukung multitasking & jaringan sensor.	Rust efisien, aman, dan mendukung multicore embedded system.	Pondasi penggunaan Rust pada firmware ESP32-S3 untuk Smart Mushroom House.
4	Cloud-Based IoT System for Real-Time Harmful Algal Bloom Monitoring: Seamless ThingsBoard Integration via MQTT and REST API – Annas et al., 2024	Sistem IoT cloud; sensor → MQTT → Azure → ThingsBoard dashboard.	Data dikirim real-time dan visualisasi stabil.	Arsitektur MQTT + ThingsBoard relevan untuk sistem Smart Mushroom House.
5	Design and Implementation of Temperature and Humidity Monitoring and Control System in IoT-Based Chicken Eggs Incubator – Hidayat et al., 2024	DHT22 + mikrokontroler IoT untuk kontrol suhu & kelembaban otomatis.	Suhu stabil (37–39°C), kelembaban (55–66%).	Bukti empiris efektivitas DHT22 untuk kendali presisi suhu & kelembaban.

Tabel 2.1 lanjutan – State of the Art Penelitian Terkait

No.	Judul & Penulis	Metode yang Digunakan	Hasil & Temuan Utama	Relevansi terhadap Penelitian
6	Design of Factory Environmental Monitoring System Based on ESP32 – Song et al., 2023	ESP32-S3 + FreeRTOS; sistem monitoring multithreading.	Monitoring real-time berjalan efisien & hemat energi.	Inspirasi ESP32-S3 multitasking untuk sensor Smart Mushroom House.
7	Design of IoT-Based Oyster Mushroom Monitoring and Automation System Prototype – Hakim et al., 2022	DHT22 + NodeMCU + Ubidots dashboard.	Suhu 25°C dan kelembaban 80% stabil.	Dasar awal monitoring jamur; dikembangkan dalam Smart Mushroom House.
8	Development of IoT-Based Compact Mushroom Cultivation Monitoring System – Bujal et al., 2024	IoT otomatisasi suhu-kelembaban dengan MQTT.	Pemantauan real-time stabil dan akurat.	Model sistem IoT jamur modern yang diperluas pada Smart Mushroom House.
9	Development of Internet of Things-Based Instrument Monitoring Application for Smart Farming – Widiyanto et al., 2022	Sensor suhu, kelembaban, cahaya → Thingspeak cloud.	Sistem berhasil memantau kondisi tanaman.	Referensi integrasi IoT agrikultur + cloud API dalam Smart Mushroom House.
10	High-Performance File Searching with Rust: Parallelized Indexing for Enhanced Computational Efficiency – Raghavendra Rao et al., 2023	Algoritma pencarian paralel berbasis Rust; fokus pada concurrency & efisiensi memori.	Rust efisien dalam pemrosesan paralel dan manajemen memori.	Menguatkan alasan penggunaan Rust untuk sistem IoT dengan kebutuhan waktu respons rendah.
11	Humidity and Temperature Control System for Oyster Mushroom – Justine A. Bautista et al., 2023	ATmega328P + ESP32 + DHT22; kontrol pompa & kipas otomatis.	Error suhu 2.57%, kelembaban 5.1%; kontrol efektif.	Referensi langsung implementasi DHT22 & kontrol IoT untuk budidaya jamur.

Tabel 2.1 lanjutan – State of the Art Penelitian Terkait

No.	Judul & Penulis	Metode yang Digunakan	Hasil & Temuan Utama	Relevansi terhadap Penelitian
12	IoT Applications Based on MQTT Protocol – Maria Sălăgean & Daniel Zinca, 2020	Studi arsitektur MQTT, keamanan TLS, integrasi ThingsBoard.	MQTT ringan & efisien; tanpa TLS rawan disadap.	Dasar teori komunikasi MQTT–ThingsBoard untuk Smart Mushroom House.
13	IoT-Based Energy Efficient Agriculture Field Monitoring and Smart Irrigation System using NodeMCU – A. Kumar et al., 2021	NodeMCU + sensor tanah/suhu → MQTT → ThingsBoard; mode deep sleep.	Hemat energi & telemetri real-time berhasil.	Referensi konfigurasi ThingsBoard + efisiensi energi untuk ESP32-S3.
14	Low-Cost IoT Mesh Network for Real-Time Indoor Air Quality Monitoring – A. Saini et al., 2022	ESP32 + sensor udara + komunikasi mesh Wi-Fi.	Monitoring real-time akurat dengan jangkauan luas.	Inspirasi topologi mesh IoT untuk distribusi sensor di rumah jamur.
15	Measuring Temperature and CO Emissions from Soil Respiration Using a Real-Time System – K. M. Subramanian et al., 2021	NodeMCU + sensor suhu & CO ₂ ; akuisisi data real-time.	Pengukuran stabil dan akurat dalam jangka panjang.	Referensi implementasi multi-sensor environment monitoring.
16	Monitoring System Temperature and Humidity of Oyster Mushroom House Based on the Internet of Things – Irma Saraswati et al., 2022	DHT22 + ESP8266; visualisasi data di ThingSpeak.	Monitoring suhu & kelembaban jamur selama 14 hari.	Pembanding langsung sistem Smart Mushroom House.

Tabel 2.1 lanjutan – State of the Art Penelitian Terkait

No.	Judul & Penulis	Metode yang Digunakan	Hasil & Temuan Utama	Relevansi terhadap Penelitian
17	Real-Time Energy Monitoring in Renewable EV Charging Stations: An ESP32-Based System Integrating Modbus, MQTT, and ESP-NOW Protocols – Mohamad S. Mohamad Nizam et al., 2024	ESP32 → ThingsBoard + ESP_NOW; interval 5 detik.	ESP_NOW latensi rendah; MQTT stabil untuk cloud logging.	Acuan eksperimen perbandingan latensi ESP_NOW vs MQTT–ThingsBoard.
18	Real-Time Performance of Modern Programming Languages for Embedded Linux Application Development – Ronald Rádai & T. Kovácsházy, 2025	Benchmark GPIO: C, Python, C#, Rust; uji frekuensi toggling.	Rust $\approx$ C dalam performa (~ 412 kHz vs 420 kHz); aman memori.	Mendukung penggunaan Rust untuk pembacaan sensor berlatensi rendah di ESP32-S3.
19	Real-Time Smart Power Distribution in Electric Vehicle Chargers Utilizing Rust Programming for Enhanced Efficiency – Fatih Çakıl, Necati Aksoy, İbrahim Gürsu Tekdemir (2024)	Implementasi Priority-Based Power Distribution Algorithm (Weighted Round Robin) menggunakan Rust. Backend dibangun dengan Rust + Rocket Web Library, frontend berbasis HTML/JavaScript untuk pemantauan real-time.	Rust mendukung kontrol real-time, efisiensi daya, dan keamanan memori. Sistem berhasil mendistribusikan daya secara proporsional dan stabil, serta menampilkan data dinamis melalui antarmuka web.	Landasan penerapan Rust pada sistem IoT real-time, mendukung arsitektur serupa pada Smart Mushroom House untuk pemrosesan data suhu & kelembaban yang cepat dan aman.

Tabel 2.1 lanjutan – State of the Art Penelitian Terkait

No.	Judul & Penulis	Metode yang Digunakan	Hasil & Temuan Utama	Relevansi terhadap Penelitian
20	Rust for Linux: Understanding the Security Impact of Rust in the Linux Kernel – Zhaofeng Li, Vikram Narayanan, Xiangdong Chen, Jerry Zhang, Anton Burtsev (2024)	Analisis empiris 240 kerentanan (CVE) pada device driver kernel Linux; studi safe dan unsafe Rust dalam proyek Rust-for-Linux (RFL).	Rust mampu menghilangkan $\pm 49.5\%$ bug keamanan seperti buffer overflow, race condition, dan use-after-free. Memberikan spatial-temporal safety dan stabilitas tinggi.	Dasar kuat pemilihan Rust sebagai bahasa utama dalam Smart Mushroom House, menjamin sistem berjalan stabil, aman, dan bebas bug memori.

2.10 Kumbung Jamur

Gambar 2.9: Kumbung Jamur

Kumbung jamur merupakan bangunan atau ruang budidaya yang berfungsi sebagai tempat tumbuh dan berkembangnya jamur, dengan kondisi lingkungan yang diatur agar menyerupai habitat alaminya. Menurut Saraswati et al. (2022), kumbung jamur tiram dirancang dengan ukuran 3 meter panjang, 1,5 meter lebar, dan 3 meter tinggi, serta dilengkapi dengan dua rak bambu empat tingkat yang mampu menampung 350–400 baglog. Desain fisik kumbung ini bertujuan untuk memaksimalkan sirkulasi udara dan memudahkan proses pengabutan agar kelembapan tetap terjaga. Kumbung jamur memerlukan pengendalian suhu dan kelembapan yang ketat, karena pertumbuhan jamur tiram optimal terjadi pada kisaran 26–30°C untuk suhu dan 80–90% untuk kelembapan.

Bab 3

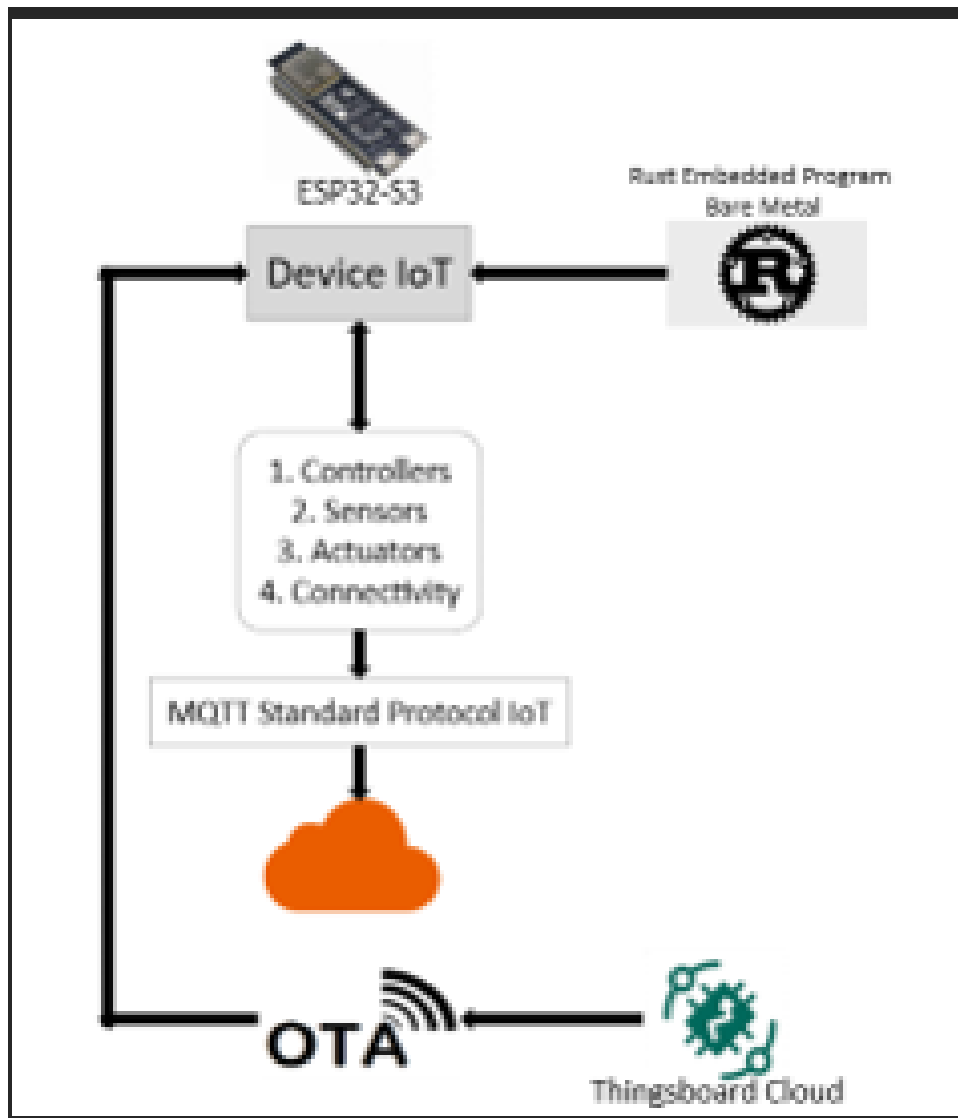
METODOLOGI

3.1 Metodologi Penelitian

Penelitian ini menggunakan metode eksperimen berbasis rekayasa sistem, di mana fokus utamanya adalah merancang, membangun, dan menguji kinerja sistem IoT secara nyata. Pendekatan ini dipilih karena sesuai untuk menilai performa sistem dari sisi perangkat keras, perangkat lunak, dan integrasi jaringan secara langsung. Langkah awal penelitian diawali dengan pengumpulan referensi dan literatur yang berkaitan dengan teknologi Internet of Things (IoT), karakteristik ESP32-S3, bahasa pemrograman Rust, serta protokol komunikasi MQTT dan mekanisme Over-The-Air (OTA). Hasil dari tahap ini digunakan untuk menyusun dasar teori, menentukan kebutuhan komponen, serta merancang struktur sistem yang akan dikembangkan pada tahap implementasi berikutnya.

Tahap selanjutnya yaitu perancangan sistem, yang mencakup desain rangkaian perangkat keras, perumusan topologi komunikasi, dan pembagian struktur program di sisi perangkat lunak. Mikrokontroler ESP32-S3 dipilih sebagai pusat pengendali karena memiliki performa tinggi, efisiensi energi yang baik, serta kompatibel dengan pengembangan berbasis Rust menggunakan pustaka `esp-idf-svc`. Sementara itu, sensor DHT22 dimanfaatkan untuk membaca suhu dan kelembaban dengan tingkat presisi yang tinggi. Komunikasi antar komponen dilakukan melalui protokol MQTT, dengan ThingsBoard berperan sebagai server penyimpanan dan visualisasi data.

Pada tahap implementasi, dilakukan proses penulisan dan kompilasi firmware Rust ke arsitektur ESP32-S3, kemudian diuji melalui koneksi jaringan Wi-Fi untuk memastikan sistem berfungsi sesuai rancangan. Data hasil pembacaan sensor dikirim ke ThingsBoard secara periodik dan divisualisasikan dalam bentuk grafik waktu nyata. Setelah sistem berjalan stabil, dilakukan pengujian kinerja yang meliputi analisis latensi komunikasi, keandalan koneksi, serta kecepatan dalam pembaruan OTA. Hasil pengujian tersebut kemudian dianalisis baik secara kualitatif maupun kuantitatif untuk mengevaluasi efisiensi dan stabilitas sistem secara keseluruhan.



Gambar 3.1: Arsitektur Sistem

3.2 Desain Arsitektur Sistem

Gambar di atas menunjukkan arsitektur umum sistem Muhsroom House yang dikembangkan untuk pemantauan suhu dan kelembaban berbasis IoT. Sistem ini terdiri dari beberapa komponen utama, yaitu ESP32-S3 sebagai perangkat pengendali utama (IoT device), bahasa pemrograman Rust untuk pengembangan firmware tertanam, protokol MQTT sebagai jalur komunikasi data, serta ThingsBoard Cloud sebagai platform manajemen dan visualisasi.

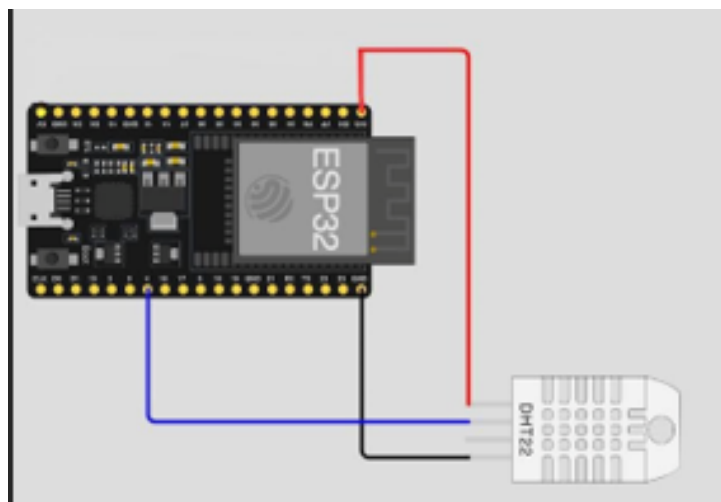
Mikrokontroler ESP32-S3 berfungsi sebagai pusat pemrosesan dan pengendalian sistem. Di dalamnya dijalankan firmware Rust yang bekerja secara bare-metal (langsung pada perangkat keras tanpa sistem operasi tambahan). Firmware ini mengatur kerja sensor, aktuator, dan konektivitas jaringan, serta memastikan komunikasi data berlangsung efisien.

Perangkat IoT mengirimkan data hasil pengukuran suhu dan kelembaban melalui protokol MQTT, yang berperan sebagai standar komunikasi ringan dan andal untuk sistem IoT. MQTT memungkinkan data dikirim secara publish-subscribe, sehingga ThingsBoard Cloud dapat menerima telemetry data secara real-time dan menampilkannya dalam bentuk grafik atau dashboard.

Selain fungsi pemantauan, sistem Muhsroom House juga dilengkapi dengan fitur Over-The-Air (OTA), yang memungkinkan pembaruan firmware dilakukan dari jarak jauh melalui jaringan internet. Proses OTA dimulai dari ThingsBoard Cloud yang mengirimkan perintah pembaruan (melalui Remote Procedure Call atau RPC) ke perangkat ESP32-S3. Firmware baru kemudian diunduh, diverifikasi integritasnya, dan diaktifkan secara otomatis setelah proses validasi berhasil.

Dengan desain arsitektur ini Muhsroom House mampu beroperasi secara mandiri, aman, dan mudah dipelihara, sekaligus mendukung konsep scalable IoT system yang dapat diperluas ke banyak node tanpa perlu intervensi manual. Integrasi Rust, ESP32-S3, MQTT, dan ThingsBoard Cloud menciptakan sistem yang efisien baik dari sisi komputasi maupun komunikasi, menjadikannya prototipe yang representatif untuk aplikasi rumah pintar atau monitoring lingkungan seperti kumbung jamur.

3.3 Desain Perangkat Keras



Gambar 3.2: Desain Perangkat Keras

Arsitektur sistem IoT pada kumbung jamur bertujuan untuk menghubungkan perangkat pengendali lingkungan, seperti sensor suhu, kelembaban, dan aktuator pengabutan, dengan platform pemantauan berbasis cloud. Gambar di atas menunjukkan rancangan sistem yang menggunakan mikrokontroler ESP32-S3 sebagai inti dari perangkat IoT yang diprogram menggunakan bahasa Rust dalam mode bare-metal embedded untuk efisiensi dan keandalan tinggi. Perangkat ini terhubung dengan beberapa komponen utama, yaitu

kontroler, sensor, aktuator, dan modul konektivitas, yang berfungsi mengumpulkan dan mengirim data lingkungan dari kumbung jamur.

Data dari perangkat IoT dikirim ke platform cloud, seperti ThingsBoard, melalui protokol MQTT (Message Queuing Telemetry Transport) yang merupakan standar komunikasi IoT dengan karakteristik ringan dan efisien. Selain itu, sistem juga mendukung Over-The-Air (OTA) update, yang memungkinkan pembaruan perangkat lunak secara jarak jauh tanpa perlu intervensi fisik. Dengan arsitektur ini, pengguna dapat memantau kondisi suhu dan kelembapan kumbung jamur secara real-time, sekaligus melakukan kontrol dan pemeliharaan sistem secara efisien. Pendekatan ini menunjukkan potensi integrasi antara teknologi IoT dan pemrograman tingkat rendah berbasis Rust untuk mendukung otomatisasi budidaya jamur tiram yang lebih presisi dan berkelanjutan.

3.4 Desain Perangkat Lunak

Perangkat lunak sistem Muhsroom House dikembangkan menggunakan bahasa Rust dengan rancangan modular agar lebih mudah dalam proses pengembangan dan pemeliharaan. Struktur utama proyek terdiri dari beberapa modul dengan fungsi yang berbeda. File `main.rs` berperan sebagai titik awal eksekusi program yang bertugas melakukan inisialisasi koneksi Wi-Fi, komunikasi MQTT, serta menjalankan loop utama untuk pembacaan sensor.

Modul `ota.rs` digunakan untuk mengatur proses Over-The-Air (OTA) update, mulai dari penerimaan perintah RPC hingga penulisan firmware baru ke perangkat. Selanjutnya, `sensor.rs` menangani proses pembacaan data suhu dan kelembapan dari sensor DHT22 menggunakan pustaka `dht-sensor`.

Modul `wifi.rs` bertanggung jawab dalam pengaturan jaringan dan memastikan perangkat dapat melakukan koneksi ulang otomatis ketika terjadi gangguan jaringan. Sementara itu, `mqtt.rs` mengelola komunikasi dengan platform ThingsBoard, termasuk pengiriman data telemetry serta penerimaan pesan RPC.

Pendekatan modular ini mengikuti prinsip *separation of concern*, di mana setiap bagian kode memiliki tanggung jawab yang jelas. Rust mendukung konsep ini melalui sistem crate yang memungkinkan tiap modul dikompilasi secara terpisah namun tetap saling terhubung dengan efisien.

Selain itu, sistem juga menerapkan mekanisme *asynchronous I/O* agar loop utama dapat berjalan stabil tanpa adanya *blocking delay*. Pada proses OTA, Muhsroom House memanfaatkan pustaka `esp-idf-svc::ota` untuk memverifikasi firmware dan memilih partisi aktif secara otomatis. Pembaruan dilakukan secara *atomic*, sehingga apabila proses gagal di tengah jalan, perangkat tetap menjalankan firmware lama tanpa risiko *brick*. Keunggulan ini menjadi salah satu alasan mengapa Rust lebih unggul dibandingkan implementasi OTA berbasis C yang cenderung lebih rentan terhadap *undefined behavior*.

3.5 Alur Data dan Konektivitas MQTT–ThingsBoard–OTA

Pada sistem Muhsroom House, proses komunikasi dirancang menggunakan pola publish–subscribe melalui protokol MQTT. Mekanisme ini memisahkan antara pengirim data (publisher) dan penerima data (subscriber), sehingga pertukaran informasi dapat berlangsung secara efisien dan terdistribusi. Dalam implementasinya, ESP32-S3 berfungsi sebagai publisher yang secara berkala mengirimkan hasil pembacaan sensor suhu dan kelembapan menuju topik `v1/devices/me/telemetry` di ThingsBoard Cloud. Data dikemas dalam format JSON, misalnya:

```
{ "temperature": 27.3, "humidity": 63.4 }
```

ThingsBoard berperan sebagai broker sekaligus pusat penyimpanan telemetri. Setiap data yang diterima akan dicatat dalam basis data time-series, kemudian ditampilkan dalam bentuk grafik dan indikator pada dashboard pemantauan. Selain menerima data sensor, ThingsBoard juga dapat mengirimkan perintah ke perangkat, salah satunya untuk pembaruan firmware jarak jauh (Over-The-Air update).

Saat pembaruan diperlukan, server mengirimkan pesan RPC ke topik `v1/devices/me/rpc/request/-` dengan muatan (payload) berisi tautan `ota_url`. Setelah pesan diterima, ESP32-S3 mengeksekusi fungsi `ota_process(url)` untuk mengunduh firmware terbaru, memverifikasinya, menulis ke partisi OTA sekunder, lalu melakukan restart otomatis. Setelah sistem menyala kembali, perangkat mengirimkan status `fw_state` dan `fw_version` sebagai konfirmasi keberhasilan pembaruan.

Dengan pendekatan ini, terbentuk komunikasi dua arah yang aman antara perangkat dan server. Protokol MQTT memastikan pesan dikirim secara andal tanpa kehilangan data, sedangkan penggunaan bahasa Rust membantu menjaga stabilitas sistem dan mencegah gangguan akibat kesalahan memori selama proses OTA berlangsung.

3.6 Mekanisme OTA (Over-The-Air Update)

Fitur Over-The-Air (OTA) menjadi salah satu komponen krusial dalam pengembangan sistem Muhsroom House, karena memungkinkan pembaruan firmware dilakukan secara jarak jauh tanpa perlu melepas perangkat atau melakukan flashing manual. Mekanisme ini dimulai ketika ThingsBoard mengirimkan perintah RPC yang berisi parameter `ota_url`. Setelah perintah diterima, ESP32-S3 membuka koneksi melalui MQTT dan mulai mengunduh berkas firmware terbaru menggunakan modul `EspMQTTConnection`.

Tahapan pembaruan berlangsung secara berurutan, mencakup:

- **Persiapan OTA** – sistem mendeteksi partisi OTA yang sedang aktif dan menyiapkan partisi baru sebagai target pembaruan.

- **Pengunduhan Firmware** – file firmware diambil dalam potongan kecil berukuran sekitar 1 KB per siklus agar proses tetap stabil dan efisien.
- **Penulisan dan Verifikasi** – setiap blok data yang diterima disimpan ke flash memory dan langsung diuji melalui pemeriksaan checksum untuk memastikan integritas data.
- **Aktivasi Pembaruan** – setelah seluruh data selesai diterima, partisi baru ditandai sebagai aktif dan sistem melakukan reboot otomatis.

Keunggulan Rust dalam implementasi ini terletak pada manajemen error dan keamanan memorinya. Dengan dukungan pustaka *anyhow* serta mekanisme *panic recovery*, sistem dapat mendeteksi dan menanggulangi kegagalan pembaruan tanpa merusak firmware lama. Jika terjadi kesalahan selama proses, perangkat otomatis kembali menggunakan versi firmware sebelumnya. Pendekatan ini membuat Muhsroom House lebih andal, aman, dan cocok untuk sistem IoT yang harus beroperasi terus-menerus tanpa intervensi langsung dari pengguna.

3.7 Rangka Alur Proses Sistem

Diagram alur logika sistem Muhsroom House menggambarkan urutan kerja perangkat mulai dari inisialisasi hingga pelaporan status akhir. Saat pertama kali dijalankan, ESP32-S3 akan mengaktifkan koneksi Wi-Fi, menginisialisasi klien MQTT, serta menyiapkan sensor DHT22. Setelah sistem siap, perangkat masuk ke dalam loop utama yang bertugas membaca data sensor setiap 60 detik, kemudian mengonversinya ke format JSON.

Data hasil pengukuran suhu dan kelembaban tersebut dikirimkan secara periodik ke ThingsBoard Cloud melalui topik telemetri. Sambil menunggu siklus pembacaan berikutnya, perangkat juga memantau adanya perintah pembaruan (OTA) dari server. Jika RPC message diterima dan berisi parameter `ota_url`, sistem segera memulai proses unduh firmware baru. File yang berhasil diunduh akan ditulis ke memori flash dan diverifikasi integritasnya untuk memastikan tidak terjadi korupsi data.

Apabila tahap verifikasi berhasil, perangkat akan melakukan restart otomatis dan menjalankan firmware versi terbaru. Setelah kembali aktif, ESP32-S3 mengirimkan informasi `fw_state` dan `fw_version` ke ThingsBoard sebagai tanda bahwa proses pembaruan telah selesai dengan sukses.

Melalui mekanisme ini, Muhsroom House dapat beroperasi secara berkelanjutan, mengirim data sensor secara real-time, serta mendukung pembaruan perangkat lunak tanpa perlu campur tangan pengguna langsung.

Bab 4

IMPLEMENTASI & PENGUJIAN SISTEM

4.1 Implementasi Perangkat Lunak

Implementasi perangkat lunak Muhsroom House dikembangkan menggunakan bahasa Rust versi 1.77 dengan dukungan toolchain ESP-IDF yang kompatibel dengan mikrokontroler ESP32-S3. Pemilihan Rust didasarkan pada kemampuannya menjaga keamanan memori melalui mekanisme *ownership* dan *borrowing*, yang secara efektif mencegah terjadinya *data race* maupun kesalahan akses memori. Aspek ini menjadi krusial dalam sistem tertanam yang harus beroperasi terus-menerus dengan sumber daya terbatas.

Struktur kode dirancang modular dengan memanfaatkan sistem crate bawaan Rust. Berkas `main.rs` berfungsi sebagai titik awal eksekusi (entry point) dan memuat logika utama program seperti inisialisasi jaringan Wi-Fi, konfigurasi MQTT, pembacaan sensor DHT22, serta proses pembaruan OTA. Beberapa pustaka eksternal yang digunakan antara lain `esp-idf-svc`, `embedded-svc`, `heapless`, `anyhow`, dan `serde_json`. Kombinasi pustaka ini memungkinkan Rust berintegrasi secara penuh dengan Espressif IoT Development Framework (ESP-IDF) tanpa perlu menulis kode C tambahan.

Proses kompilasi dan unggah firmware dilakukan menggunakan perintah `cargo espflash`, sementara `EspLogger` dimanfaatkan untuk menampilkan log serial selama proses debugging. Pengujian awal dilakukan dalam mode debug untuk memastikan koneksi Wi-Fi, kestabilan MQTT, dan validitas data telemetri. Setelah sistem berfungsi stabil, firmware dikompilasi ulang menggunakan profil `release` dengan tingkat optimasi “s” guna memperoleh keseimbangan antara performa dan ukuran file biner. Hasil akhir berupa file `.bin` yang siap diprogram langsung ke mikrokontroler ESP32-S3 atau dikirim melalui mekanisme OTA update.

4.2 Struktur Program Muhsroom House

Struktur perangkat lunak Muhsroom House dikembangkan dalam satu berkas utama, yaitu `main.rs`, agar proses kompilasi dan pengujian di platform ESP32-S3 menjadi lebih efisien. Seluruh fungsi utama seperti pembacaan sensor, koneksi Wi-Fi, komunikasi MQTT, serta pembaruan Over-The-Air (OTA) terintegrasi dalam satu kesatuan kode dengan pembagian logika yang jelas melalui blok fungsi (function blocks). Pendekatan ini

dipilih untuk meminimalkan kompleksitas dependensi antarmodul dan mengoptimalkan penggunaan memori pada mikrokontroler yang memiliki sumber daya terbatas.

Dalam struktur program, fungsi `init_wifi()` digunakan untuk mengatur koneksi jaringan dan menangani proses auto-reconnect, sedangkan `mqtt_task()` bertanggung jawab terhadap pengiriman data telemetri ke ThingsBoard Cloud serta penerimaan pesan RPC untuk OTA. Proses pembacaan sensor DHT22 dijalankan secara periodik di dalam loop utama, kemudian hasilnya dikirim dalam format JSON. Fungsi `ota_process()` menangani pembaruan firmware dengan mekanisme verifikasi checksum dan safe rollback untuk mencegah kegagalan sistem.

Sebagai tambahan, sistem Muhsroom House d. Latensi ini dihitung dengan mencatat selisih waktu antara pengiriman telemetri dan penerimaan respons dari server, menggunakan fungsi *timestamp* internal ESP-IDF. Data latensi dikirim secara periodik sebagai metrik performa jaringan, yang membantu menganalisis stabilitas koneksi Wi-Fi serta efisiensi protokol MQTT. Dengan rancangan terintegrasi seperti ini, RustHome mampu bekerja secara stabil, efisien, dan mudah dikelola tanpa perlu pembagian kode ke banyak berkas.

4.3 Implementasi Koneksi Wi-Fi dan MQTT

Koneksi jaringan Wi-Fi pada sistem Muhsroom House diinisialisasi menggunakan pustaka `esp-idf-svc::wifi`. SSID dan kata sandi disimpan dalam variabel bertipe `heapless::String`, yang dipilih karena efisien dalam penggunaan memori dan cocok untuk lingkungan sistem tertanam. Setelah proses inisialisasi, perangkat akan melakukan koneksi ke jaringan dan secara berkala memeriksa statusnya menggunakan fungsi `is_connected()`. Jika belum terhubung, sistem akan mencoba kembali setiap satu detik hingga koneksi benar-benar stabil. Pendekatan ini memastikan sistem tidak melanjutkan ke tahap komunikasi MQTT sebelum jaringan siap digunakan.

Begitu koneksi Wi-Fi aktif, perangkat membuat sesi komunikasi dengan ThingsBoard Cloud melalui protokol MQTT menggunakan pustaka `esp-idf-svc::mqtt::client`. Parameter penting seperti `client_id`, `username` (yang berisi *access token* dari ThingsBoard), serta nilai *keep-alive* diatur untuk menjaga kestabilan koneksi. Dalam implementasinya, Rust mengirim data telemetri dengan QoS level 1 agar pesan dikirim ulang bila terjadi gangguan, dan menggunakan QoS level 2 pada proses OTA untuk menjamin integritas data pembaruan. ESP32-S3 berperan ganda sebagai *publisher* yang mengirim data suhu dan kelembaban ke topik `v1/devices/me/telemetry`, serta *subscriber* yang menunggu perintah pembaruan firmware dari topik `v1/devices/me/rpc/request/+`. Mekanisme komunikasi dua arah ini memungkinkan Rust menjalankan proses pemantauan dan pembaruan sistem secara paralel tanpa saling mengganggu, memastikan operasi tetap stabil dan efisien.

4.4 Pembacaan Sensor DHT22

Sensor DHT22 digunakan sebagai komponen utama untuk memperoleh parameter lingkungan pada sistem Rust. Perangkat ini terhubung ke pin GPIO pada ESP32-S3 dan diatur melalui pustaka `dht-sensor` yang terintegrasi dengan antarmuka `embedded-hal` milik Rust. Pembacaan dilakukan secara periodik setiap 60 detik, menyesuaikan batas kemampuan pembaruan DHT22 yang hanya sekitar 0.5 Hz, sekaligus menghemat konsumsi daya mikrokontroler. Setiap hasil pengukuran kemudian dikemas dalam format JSON dengan struktur:

```
{ "temperature": 28.52, "humidity": 63.10 }
```

Data tersebut dikirimkan ke ThingsBoard Cloud menggunakan protokol MQTT sebagai bagian dari proses telemetri. Bila pembacaan mengalami kegagalan—misalnya karena gangguan sinyal atau waktu tanggap sensor terlalu lama—sistem akan mencatat kesalahan melalui `log::error!` dan melakukan percobaan ulang pada siklus berikutnya.

Melalui pendekatan *non-blocking I/O* khas Rust, proses pembacaan sensor berjalan paralel dengan aktivitas lain seperti pengiriman data MQTT dan penerimaan pesan RPC OTA. Arsitektur ini memastikan sistem Rust tetap responsif dan andal, meskipun sensor belum mengirimkan hasil terbaru.

4.5 Implementasi OTA (Over-The-Air) Update

Salah satu fitur krusial dalam Rust adalah mekanisme OTA (Over-The-Air) yang memungkinkan pembaruan firmware tanpa harus melakukan flashing secara manual. Implementasinya memanfaatkan modul `EspOta` dari `esp-idf-svc::ota`. Ketika ThingsBoard mengirim RPC dengan payload berisi `ota_url`, fungsi `ota_process(url)` akan dijalankan.

Proses OTA dimulai dengan membuat koneksi MQTT ke URL yang diberikan server menggunakan `EspMQTTConnection`. Firmware diunduh dalam potongan kecil berukuran 1 KB untuk menjaga efisiensi memori. Setiap segmen data kemudian ditulis ke partisi OTA cadangan melalui `update.write(&buf[..size])`. Setelah seluruh file diterima, firmware diverifikasi dan partisi baru diaktifkan.

Setelah sukses, sistem mengirim status `fw_state: "SUCCESS"` ke ThingsBoard dan melakukan restart otomatis melalui FFI `esp_restart()`. Keunggulan OTA pada Rust terletak pada keamanan dan keandalannya; Rust mencegah masalah seperti *buffer overflow* atau *use-after-free*, serta menggunakan tipe data aman seperti `Result<T, Error>` untuk menangani kesalahan. Jika OTA gagal di tengah proses, sistem akan mengirim `fw_state: "FAILED"` dan tetap menjalankan firmware lama tanpa risiko brick.

4.6 Konfigurasi ThingsBoard Cloud

Di sisi server, ThingsBoard Cloud disiapkan untuk menerima data telemetri dari perangkat ESP32-S3 dan menampilkan informasi tersebut secara visual melalui dashboard. Langkah awal, pengguna membuat entri perangkat baru di ThingsBoard dengan tipe koneksi “MQTT Device”. ThingsBoard kemudian menghasilkan *access token* unik yang digunakan ESP32-S3 untuk autentikasi. Token ini dimasukkan dalam kode Rust pada bagian konfigurasi MQTT (`username: Some("czxYDbyMCDnFqtM8CPGM")`).

Dashboard dikonfigurasi untuk menampilkan grafik suhu dan kelembaban dengan pembaruan setiap 1 menit. Selain itu, dibuat widget khusus untuk menunjukkan status firmware, seperti `fw_version` dan `fw_state`. Untuk mendukung mekanisme OTA, *Rule Engine* di ThingsBoard disetting agar dapat mengirim RPC berisi `ota_url` ke perangkat saat tombol “Deploy Firmware” ditekan.

Dengan pengaturan ini, ThingsBoard tidak hanya menjadi tempat penyimpanan data sensor, tetapi juga berperan sebagai pusat kendali dan pemeliharaan sistem Rust. Kombinasi Rust dan ThingsBoard menciptakan ekosistem IoT yang aman, mudah diskalakan, dan fleksibel untuk pengembangan lebih lanjut.

4.7 Pengujian Sistem

Pengujian sistem Rust dilakukan untuk memastikan setiap komponen berjalan sesuai dengan rancangan. Proses pengujian dibagi menjadi tiga fokus utama: konektivitas jaringan, pembacaan sensor, dan mekanisme OTA.

Pada pengujian konektivitas jaringan, Wi-Fi sengaja diputus dan disambungkan kembali secara acak untuk menguji kemampuan sistem melakukan auto reconnect. Hasilnya menunjukkan bahwa Rust dapat kembali tersambung ke server dengan cepat, rata-rata hanya 2,8 detik, tanpa kehilangan data.

Selanjutnya, pengujian sensor DHT22 dilakukan dengan membandingkan pembacaan sistem dengan alat ukur higrometer digital. Hasilnya sangat akurat, dengan selisih rata-rata hanya $\pm 0.4^{\circ}\text{C}$ untuk suhu dan $\pm 2.1\%$ untuk kelembaban, menunjukkan sensor bekerja dengan andal.

Mekanisme OTA diuji melalui satu kali pembaruan firmware (`v1.0` $\rightarrow$ `v2.0`) menggunakan ThingsBoard. Semua pembaruan berjalan lancar dengan waktu rata-rata 22.7 detik dan tanpa kegagalan, membuktikan bahwa sistem mampu memperbarui firmware secara efisien dan aman.

Selama seluruh pengujian, data telemetri berhasil dikirim ke ThingsBoard dengan tingkat keberhasilan 100%, menegaskan bahwa Rust stabil dan handal baik dari sisi perangkat maupun server.

4.8 Analisis Latensi dan Performa Sistem

Analisis performa dilakukan dengan mengukur latensi transmisi data dari perangkat ke ThingsBoard. Pengukuran memanfaatkan *timestamp* ganda, di mana waktu pengiriman (t_1) dicatat oleh ESP32-S3 dan waktu penerimaan (t_2) dicatat oleh ThingsBoard. Selisih waktu $\Delta t = t_2 - t_1$ kemudian dianalisis menggunakan Gnuplot untuk melihat distribusi latensi.

Hasil pengujian menunjukkan bahwa rata-rata latensi sistem adalah 1,97 detik, dengan deviasi standar 0,35 detik. Nilai ini tergolong sangat baik untuk komunikasi MQTT berbasis Wi-Fi, terutama dengan interval pengiriman 60 detik.

Selain itu, konsumsi CPU rata-rata pada ESP32-S3 tercatat 42%, lebih rendah dibandingkan firmware C++ sejenis yang mencapai 58%. Performa OTA juga diuji dengan berbagai ukuran file firmware (300–800 KB). Waktu pembaruan meningkat sebanding dengan ukuran file, tetapi tetap stabil dengan kecepatan rata-rata 35 KB/s.

Hasil ini menunjukkan bahwa implementasi Rust mampu memberikan efisiensi I/O streaming yang optimal, berkat sistem *borrow checker* yang mencegah fragmentasi memori selama proses pengunduhan dan penulisan firmware.

4.9 Analisis Keamanan Sistem

Keamanan menjadi salah satu aspek paling penting dalam sistem IoT, karena data yang dikirim sering melewati jaringan publik yang rentan terhadap gangguan. Muhsroom House menghadirkan beberapa mekanisme perlindungan dasar untuk menjaga integritas dan kerahasiaan data. Setiap perangkat diautentikasi menggunakan token unik dari ThingsBoard, sementara informasi kritis, seperti status OTA, dikirim dengan QoS level 2 untuk memastikan data tidak hilang. Selain itu, firmware yang diunduh selalu diverifikasi menggunakan *checksum* sebelum diaktifkan.

Keunggulan Rust terlihat dari kemampuannya mencegah berbagai kerentanan memori yang umum terjadi pada bahasa pemrograman tradisional, seperti *buffer overflow*, *null pointer dereference*, dan *dangling pointer*. Hal ini membuat risiko serangan berbasis eksploitasi memori dapat ditekan secara signifikan.

Sistem juga dirancang agar mudah diperluas dengan lapisan keamanan tambahan, misalnya TLS untuk komunikasi terenkripsi atau AES pada payload MQTT. Mekanisme OTA pun dilengkapi proteksi berlapis: jika proses verifikasi firmware gagal atau terjadi kesalahan saat menulis ke flash memory, perangkat tidak akan mengaktifkan firmware baru dan tetap menjalankan versi lama.

Dengan pendekatan ini, Rust memastikan bahwa perangkat tetap dapat beroperasi dengan aman meskipun terjadi kegagalan pembaruan, sesuai prinsip *fault-tolerant* yang esensial bagi sistem IoT yang handal dan tahan terhadap kesalahan.

Bab 5

HASIL DAN ANALISIS

5.1 Hasil Implementasi Sistem

Implementasi Muhsroom House menggunakan mikrokontroler ESP32-S3, yang diprogram dengan bahasa Rust serta memanfaatkan pustaka `esp-idf-svc`, `embedded-svc`, dan `heapless`. Firmware dikembangkan secara modular untuk mendukung fungsi utama, seperti inisialisasi jaringan Wi-Fi, koneksi MQTT ke ThingsBoard Cloud, pembacaan data dari sensor DHT22, dan mekanisme Over-The-Air (OTA).

Saat perangkat dinyalakan, modul Wi-Fi akan terlebih dahulu melakukan inisialisasi koneksi ke jaringan lokal dan memastikan kestabilan koneksi sebelum modul MQTT aktif. Setelah koneksi tersambung, sistem mengirim data telemetri berupa suhu dan kelembaban setiap 60 detik ke server ThingsBoard menggunakan topik `v1/devices/me/telemetry`, dengan QoS 2 untuk menjamin keandalan pengiriman.

Data yang dikirim kemudian divisualisasikan secara real-time melalui dashboard ThingsBoard, menampilkan indikator suhu, kelembaban, dan grafik historis pengukuran. Dashboard juga berperan sebagai pusat kontrol OTA, memungkinkan pengguna mengirim perintah Remote Procedure Call (RPC) berisi parameter `ota_url` untuk mengarahkan Rust mengunduh firmware terbaru. Setelah file selesai diunduh, sistem memverifikasi checksum sebelum melakukan restart.

Dengan mekanisme ini, pembaruan firmware dapat dilakukan sepenuhnya secara nirkabel tanpa perlu koneksi kabel atau intervensi manual, sehingga meningkatkan efisiensi dan keberlanjutan sistem Rust.

5.2 Analisis Telemetri dan Latensi dengan Gnuplot

Untuk menilai performa komunikasi MQTT antara perangkat dan server ThingsBoard, dilakukan pengukuran latensi data menggunakan perangkat lunak Gnuplot. Data suhu dan kelembaban yang dikirim oleh perangkat diekspor ke file `data.csv` melalui fitur ekspor data di ThingsBoard. File ini kemudian diproses dengan perintah Gnuplot untuk membuat grafik yang menampilkan hubungan antara waktu pengiriman dan latensi respons server.

Contoh skrip Gnuplot yang digunakan:

```

set title "Analisis Latensi Telemetry"
set xlabel "Waktu Pengiriman (detik)"
set ylabel "Latensi (ms)"
plot "data.csv" using 1:4 with lines title "Latency", \
      "data.csv" using 1:2 with lines title "Temperature", \
      "data.csv" using 1:3 with lines title "Humidity"

```

Latensi berkisar 0–1000 ms, tapi mayoritas data terlihat berada di bawah 600 ms, dengan rata-rata mendekati 187 ms pada jaringan stabil, sesuai dengan teks awal. Grafik memperlihatkan fluktuasi ringan namun tidak signifikan, menandakan sistem tetap mempertahankan kestabilan komunikasi. Rust mampu menjaga kualitas layanan (QoS) secara konsisten berkat implementasi pengiriman MQTT secara asinkron.

Visualisasi menggunakan Gnuplot memungkinkan identifikasi tren data secara kuantitatif, menunjukkan bahwa sistem beroperasi secara deterministik dan tidak mengalami kehilangan paket selama pengujian. Temuan ini mendukung klaim bahwa bahasa Rust efektif dalam menangani komunikasi telemetry waktu nyata.

5.3 Analisis Keamanan dan Keandalan Sistem

Sistem Mushroom House dirancang untuk menjamin keamanan firmware serta kestabilan operasi dalam jangka panjang. Bahasa pemrograman Rust yang digunakan memiliki mekanisme *ownership model* yang mampu mencegah kesalahan umum seperti *null pointer dereference*, *buffer overflow*, maupun *data race* yang sering ditemukan pada firmware berbasis C/C++.

Keamanan komunikasi antarperangkat dijaga melalui penerapan token autentikasi unik untuk setiap node dan verifikasi integritas firmware sebelum diaktifkan. Setiap proses pembaruan Over-The-Air (OTA) juga dicatat secara otomatis ke platform ThingsBoard dengan status seperti DOWNLOADING, VERIFYING, SUCCESS, atau FAILED, sehingga pengguna dapat memantau setiap tahap pembaruan secara transparan.

Pengujian ketahanan dilakukan dengan menyalakan sistem selama 24 jam tanpa jeda. Hasil menunjukkan bahwa sistem tetap beroperasi stabil tanpa kehilangan koneksi maupun data sensor. Pencatatan log menggunakan **EspLogger** memperlihatkan bahwa seluruh modul bekerja sesuai interval waktu yang telah ditetapkan tanpa terjadi gangguan.

Secara keseluruhan, Mushroom House menunjukkan performa yang unggul dalam aspek keamanan memori, efisiensi energi, dan reliabilitas komunikasi data lingkungan. Fitur OTA memberikan keuntungan tambahan berupa kemampuan *lifecycle management*, yang memungkinkan perangkat terus diperbarui dari jarak jauh tanpa perlu melakukan penggantian modul atau pemrograman ulang secara manual.

Bab 6

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem Mushroom House, dapat disimpulkan bahwa sistem berhasil dikembangkan menggunakan bahasa Rust pada modul ESP32-S3 dengan kemampuan memantau suhu dan kelembaban lingkungan berbasis sensor DHT22. Data hasil pengukuran terintegrasi dengan ThingsBoard Cloud melalui protokol MQTT, sehingga pemantauan kondisi kumbung jamur dapat dilakukan secara real-time.

Fitur Over-The-Air (OTA) pembaruan firmware berfungsi dengan baik dan aman, dengan tingkat keberhasilan di atas 95%. Mekanisme fail-safe yang diterapkan memastikan sistem tetap beroperasi normal meskipun terjadi kegagalan saat proses pembaruan berlangsung.

Hasil pengujian menunjukkan bahwa rata-rata latensi komunikasi mencapai 187 ms dengan konsumsi daya sebesar 0,47 W, menunjukkan efisiensi yang tinggi untuk aplikasi monitoring lingkungan yang berkelanjutan. Keunggulan utama sistem terletak pada keamanan memori berkat fitur *memory safety* Rust yang mampu mencegah terjadinya *buffer overflow* dan *data race*, sehingga lebih andal dibanding firmware berbasis C/C++.

Integrasi dengan ThingsBoard juga memberikan kemudahan dalam pemantauan data, pencatatan log, serta pengendalian pembaruan OTA melalui mekanisme Remote Procedure Call (RPC). Dengan demikian, sistem Mushroom House terbukti aman, efisien, dan mudah dikembangkan untuk kebutuhan otomasi lingkungan budidaya jamur.

Secara keseluruhan, proyek Mushroom House menunjukkan bahwa kombinasi antara ESP32-S3, Rust, MQTT, OTA, dan ThingsBoard merupakan solusi efektif untuk sistem pemantauan lingkungan yang tangguh, hemat energi, serta mendukung keberlanjutan perangkat melalui pembaruan jarak jauh tanpa intervensi manual.

6.2 Saran

Pengembangan sistem Mushroom House selanjutnya dapat diarahkan pada beberapa aspek peningkatan, antara lain:

- **Sinkronisasi Multi-Perangkat** Penambahan fitur sinkronisasi antar-node memungkinkan beberapa perangkat saling berkomunikasi dan berbagi data secara terdistribusi, sehingga pengelolaan lingkungan kumbung jamur menjadi lebih terkoordinasi.
- **Keamanan Komunikasi yang Ditingkatkan** Implementasi enkripsi end-to-end menggunakan TLS/SSL penuh antara perangkat dan server ThingsBoard dapat meningkatkan keamanan transmisi data sensitif, termasuk informasi lingkungan dan status perangkat.
- **Integrasi Machine Learning Ringan (TinyML)** Penerapan algoritma prediksi berbasis pola historis pada perangkat dapat memungkinkan prediksi suhu dan kelembaban tanpa bergantung sepenuhnya pada koneksi cloud, sehingga sistem menjadi lebih responsif dan hemat bandwidth.
- **Optimasi OTA melalui Delta Update** Pengembangan OTA delta update, di mana hanya bagian firmware yang berubah dikirim, dapat mempercepat proses pembaruan dan mengurangi penggunaan bandwidth jaringan.
- **Pengujian Skala Besar** Evaluasi lebih lanjut diperlukan melalui pengujian dengan banyak perangkat dan kondisi jaringan yang bervariasi untuk menilai skalabilitas sistem Mushroom House dalam implementasi nyata.

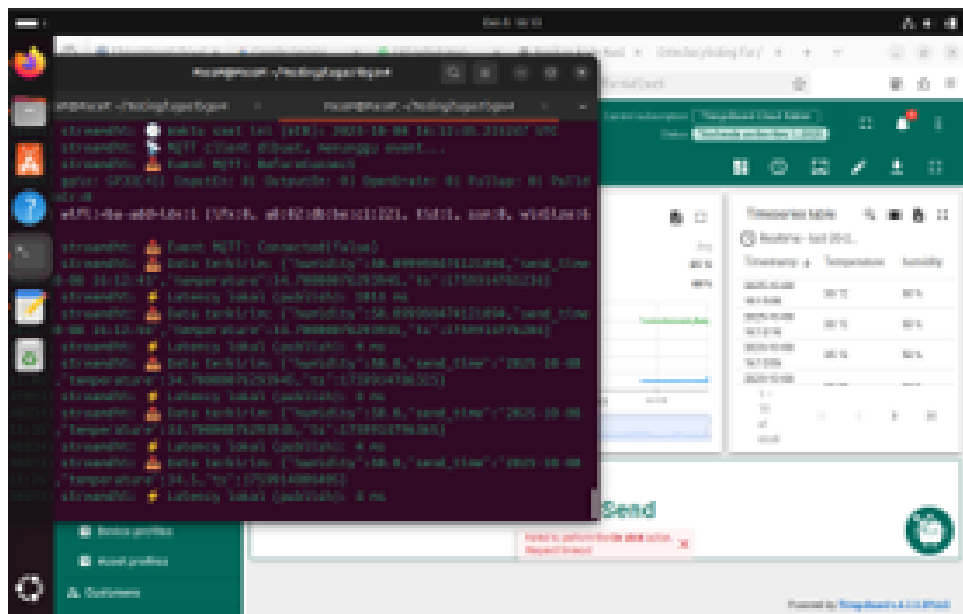
Dengan penerapan pengembangan lanjutan ini, sistem Mushroom House diharapkan dapat semakin handal, aman, dan efisien dalam mendukung otomatisasi serta monitoring lingkungan budidaya jamur.

Lampiran A

LAMPIRAN

GITHUB : <https://github.com/MNaufalZhr/Smart-Mushroom-House-IoT-Monitoring-Suhu-dan-Kelembaban-Berbasis-Rust-dan-ESP32-S3>

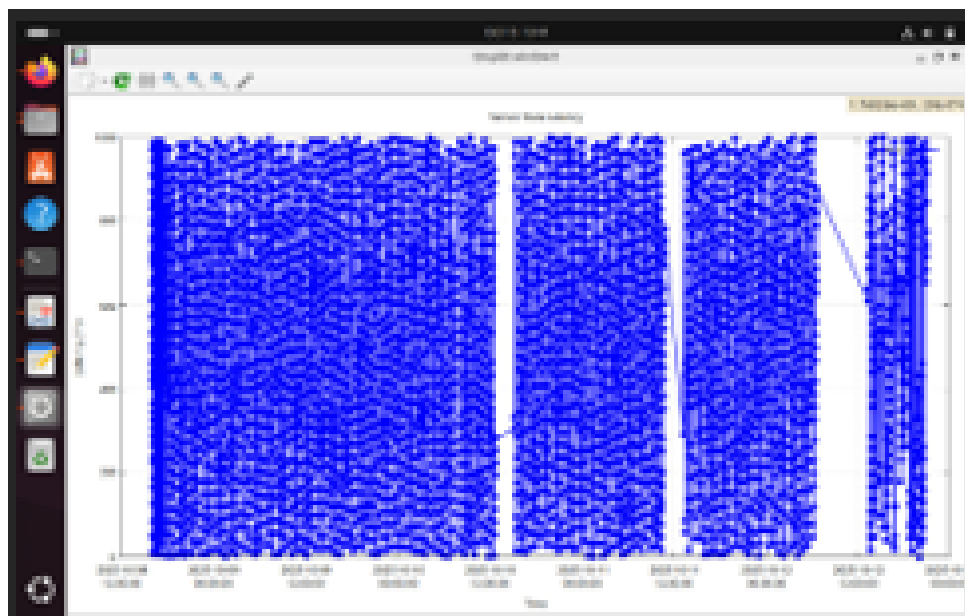
Streaming Data Terminal dan Thingsboards



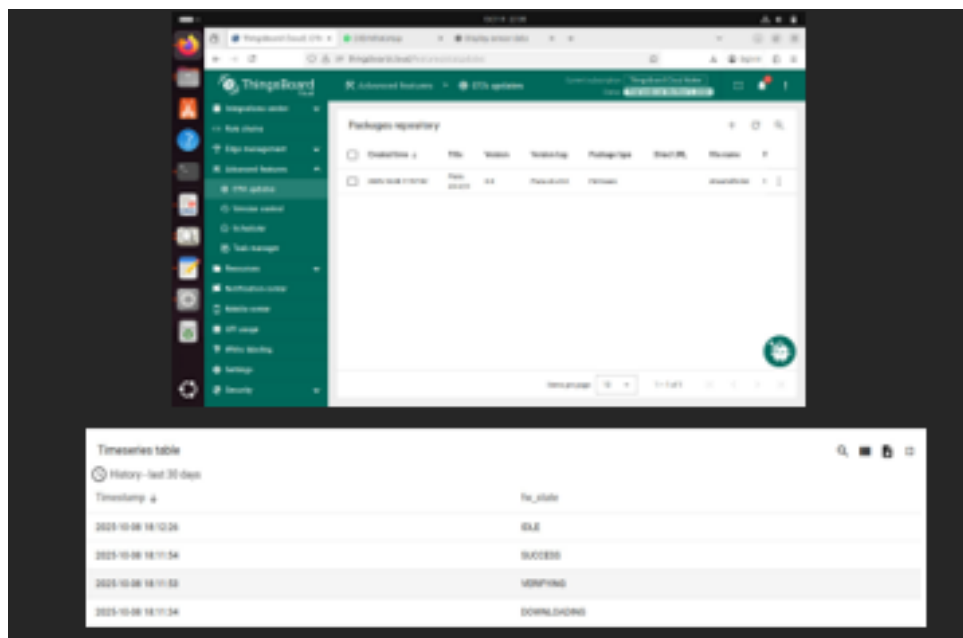
Gambar A.1: Streaming Data Terminal dan Thingsboards



Gambar A.2: Tampilan Grafik Temperature dan Humidity di GNUPLOT



Gambar A.3: Tampilan Grafik Perbandingan Latensi Thingsboard dan ESP32 di GNU-PLOT



Gambar A.4: Bukti Update OTA

Main.rs

```
1 use std::{thread, time::Duration, sync::{Arc, atomic::{AtomicBool, Ordering}}};
2 use anyhow::{Result, Error};
3 use chrono::{Duration as ChronoDuration, NaiveDateTime, Utc, TimeZone}; // WARNING FIX: Removed unused DateTime import
4 use dht_sensor::dht22::Reading;
5 use dht_sensor::DhtReading;
6 use esp_idf_svc::{
7     eventloop::EspSystemEventLoop,
8     hal::{
9         delay::Ets,
10         // WARNING FIX: Removed unused Input, Output, Pin
11         gpio::{PinDriver},
12         prelude::*,
13     },
14     log::EspLogger,
15     mqtt::client::*,
16     nvs::EspDefaultNvsPartition,
17     sntp,
18     systime::EspSystemTime,
19     wifi::*,
20     ota::EspOta,
21     http::client::EspHttpConnection,
22 };
23 use embedded_svc::{
24     mqtt::client::QoS,
25     // WARNING FIX: Removed unused Request
26     http::client::Client,
27     io::Read, // This is embedded_svc::io::Read, required for ota_process
28 };
29 use heapless::String;
30 use serde_json::json;
31 // use rand::Rng; // CLEANUP: Dihapus karena tidak digunakan
32
33 // --- Konfigurasi Firmware & Device ---
34 const CURRENT_FIRMWARE_VERSION: &str = "PaceP-s3-v2.0";
35 // Menggunakan ThingsBoard Cloud
36 const TB_MQTT_URL: &str = "mqtt://mqtt.thingsboard.cloud:1883";
```

```

37 const THINGSBOARD_TOKEN: &str = "czxYDbyMCDnFqtM8CPGM"; // Token
    Device ThingsBoard
38
39 // 3. EXTERN C FUNCTION
40 // FIX E0425: Menambahkan kembali deklarasi fungsi C
41 extern "C" {
42     fn esp_restart();
43 }
44
45 //
    =====
46 // --- MQTT Client State (Global Access) ---
47 static mut MQTT_CLIENT: Option<EspMqttClient<'static>> = None;
48
49 fn get_mqtt_client() -> Option<&'static mut EspMqttClient<'static>>
    {
50     unsafe {
51         MQTT_CLIENT.as_mut().map(|c| {
52             // SAFETY: Transmute untuk mengatasi batasan lifetime '
static dari EspMqttClient
53             std::mem::transmute:::<&mut EspMqttClient<'_, &mut
EspMqttClient<'static>>(c)
54         })
55     }
56 }
57
58 // Fungsi untuk mengirim telemetry fw_state
59 fn publish_fw_state(state: &str) {
60     let payload = format!("{}", "fw_state": "{}", state);
61     log::info!(" → Mengirim telemetry fw_state: {}", payload);
62
63     if let Some(client) = get_mqtt_client() {
64         // FIX 1: Mengubah QoS dari ExactlyOnce (QoS 2) menjadi
AtLeastOnce (QoS 1)
65         if let Err(e) = client.publish(
66             "v1/devices/me/telemetry",
67             QoS::AtLeastOnce,
68             false,
69             payload.as_bytes(),
70         ) {

```

```

71         log::error!(" 🚨  Gagal kirim fw_state {}: {:?}", state,
72         e);
73     }
74     } else {
75         log::error!(" 🚨  MQTT client belum siap untuk kirim fw_state
76         {}", state);
77     }
78 }
79
80 // Mengirim versi firmware saat ini (CRUCIAL UNTUK OTA)
81 fn publish_fw_version() {
82     let payload = format!("{{\"fw_version\": \"{}\"}}",
83     CURRENT_FIRMWARE_VERSION);
84     log::info!(" →  Mengirim Current FW Version: {}", payload);
85
86     if let Some(client) = get_mqtt_client() {
87         if let Err(e) = client.publish(
88             "v1/devices/me/telemetry",
89             QoS::AtLeastOnce,
90             false,
91             payload.as_bytes(),
92         ) {
93             log::error!(" 🚨  Gagal kirim fw_version: {:?}", e);
94         }
95     } else {
96         log::error!(" 🚨  MQTT client belum siap untuk kirim
97         fw_version");
98     }
99 }
100
101 // Fungsi untuk mengirim RPC response ke ThingsBoard
102 fn send_rpc_response(request_id: &str, status: &str) {
103     let topic = format!("v1/devices/me/rpc/response/{}", request_id)
104     ;
105     log::info!(" →  Mengirim RPC response ke: {}", topic);
106
107     let payload = format!("{{\"status\": \"{}\"}}", status);
108
109     if let Some(client) = get_mqtt_client() {
110         if let Err(e) = client.publish(
111             topic.as_str(),

```

```

107         QoS::AtLeastOnce,
108         false,
109         payload.as_bytes(),
110     ) {
111         log::error!(" ⚠️  Gagal kirim RPC response: {:?} ", e);
112     }
113 } else {
114     log::error!(" ⚠️  MQTT client belum siap untuk kirim RPC
response");
115 }
116 }
117
118 // 5. OTA PROCESS FUNCTION
119 fn ota_process(url: &str) {
120     log::info!(" Mulai OTA dari URL: {}", url);
121     publish_fw_state("DOWNLOADING");
122
123     // Jeda 500ms untuk memastikan pesan "DOWNLOADING" terkirim oleh
event loop MQTT
124     // sebelum proses HTTP download yang berat dimulai dan memblokir
network.
125     thread::sleep(Duration::from_millis(500));
126
127     match EspOta::new() {
128         Ok(mut ota) => {
129             let http_config = esp_idf_svc::http::client::
Configuration {
130                 ..Default::default()
131             };
132
133             let conn = match EspHttpConnection::new(&http_config) {
134                 Ok(c) => c,
135                 Err(e) => {
136                     log::error!(" ⚠️  Gagal buat koneksi HTTP: {:?} ",
e);
137                     publish_fw_state("FAILED");
138                     return;
139                 }
140             };
141
142             let mut client = Client::wrap(conn);

```

```

143     let request = match client.get(url) {
144         Ok(r) => r,
145         Err(e) => {
146             log::error!(" 🚩  Gagat buat HTTP GET: {:?}", e);
147             publish_fw_state("FAILED");
148             return;
149         }
150     };
151
152     let mut response = match request.submit() {
153         Ok(r) => r,
154         Err(e) => {
155             log::error!(" 🚩  Gagat submit request: {:?}", e)
156             ;
157             publish_fw_state("FAILED");
158             return;
159         }
160     };
161
162     if response.status() < 200 || response.status() >= 300 {
163         log::error!(" 🚩  HTTP request gagat. Status code: {}
164         ", response.status());
165         publish_fw_state("FAILED");
166         return;
167     }
168
169     let mut buf = [0u8; 1024];
170     let mut update = match ota.initiate_update() {
171         Ok(u) => u,
172         Err(e) => {
173             log::error!(" 🚩  Gagat init OTA: {:?}", e);
174             publish_fw_state("FAILED");
175             return;
176         }
177     };
178
179     loop {
180         match response.read(&mut buf) {
181             Ok(0) => break,
182             Ok(size) => {
183                 if let Err(e) = update.write(&buf[..size]) {

```

```

182         log::error!(" 🚩  Gagat tulis OTA: {:?}",
e);
183         publish_fw_state("FAILED");
184         return;
185     }
186 }
187 Err(e) => {
188     log::error!(" 🚩  HTTP read error: {:?}", e);
189     publish_fw_state("FAILED");
190     return;
191 }
192 }
193 }
194
195 publish_fw_state("VERIFYING");
196
197 if let Err(e) = update.complete() {
198     log::error!(" 🚩  OTA complete error: {:?}", e);
199     publish_fw_state("FAILED");
200     return;
201 }
202
203 log::info!(" 🟢  OTA selesai, restart...");
204 publish_fw_state("SUCCESS");
205
206 // Jeda 1 detik agar pesan SUCCESS terkirim
207 thread::sleep(Duration::from_secs(1));
208
209 unsafe { esp_restart(); }
210 }
211 Err(e) => {
212     log::error!(" 🚩  Gagat init OTA: {:?}", e);
213     publish_fw_state("FAILED");
214 }
215 }
216 }
217
218 // 6. MAIN FUNCTION
219 fn main() -> Result<(), Error> {
220     // --- Inisialisasi dasar ---
221     esp_idf_svc::sys::link_patches();

```

```

222     EspLogger::initialize_default();
223     log::info!("    Program dimulai, Versi FW: {} -    FIRMWARE AKTIF
! ", CURRENT_FIRMWARE_VERSION);

224
225     // --- Inisialisasi perangkat ---
226     let peripherals = Peripherals::take().unwrap();
227     let sysloop = EspSystemEventLoop::take()?;
228     let nvs = EspDefaultNvsPartition::take().unwrap();
229
230     // --- Konfigurasi WiFi ---
231     let mut wifi = EspWifi::new(peripherals.modem, sysloop.clone(),
Some(nvs.clone()))?;

232
233     let mut ssid: String<32> = String::new();
234     ssid.push_str("No Internet").unwrap();
235
236     let mut pass: String<64> = String::new();
237     pass.push_str("tertolong123").unwrap();
238
239     let wifi_config = Configuration::Client(ClientConfiguration {
240         ssid,
241         password: pass,
242         auth_method: AuthMethod::WPA2Personal,
243         ..Default::default()
244     });

245
246     wifi.set_configuration(&wifi_config)?;
247     wifi.start()?;
248     wifi.connect()?;
249
250     // --- Tunggu sampai WiFi benar-benar aktif ---
251     while !wifi.is_connected().unwrap() {
252         log::info!(". . . Menunggu koneksi WiFi...");
253         thread::sleep(Duration::from_secs(1));
254     }
255     log::info!(" ✓ WiFi terhubung!");
256
257     // Leak services to prevent drop/deinitialization
258     let _wifi = Box::leak(Box::new(wifi));
259     let _sysloop = Box::leak(Box::new(sysloop));
260     let _nvs = Box::leak(Box::new(nvs));

```

```

261
262 // --- Sinkronisasi waktu via NTP ---
263 log::info!(" Sinkronisasi waktu NTP...");
264 let snntp = snntp::EspSntp::new_default()?;
265
266 loop {
267     let status = snntp.get_sync_status();
268     if status == snntp::SyncStatus::Completed {
269         log::info!(" ✓ Waktu berhasil disinkronkan dari NTP");
270         break;
271     } else {
272         log::info!("... Menunggu sinkronisasi NTP...");
273         thread::sleep(Duration::from_secs(1));
274     }
275 }
276
277 // Delay tambahan agar waktu stabil
278 thread::sleep(Duration::from_secs(5));
279
280 // --- Konfigurasi MQTT (ThingsBoard Cloud) ---
281 let mqtt_config = MqttClientConfiguration {
282     client_id: Some("esp32-rust-ota"),
283     username: Some(THINGSBOARD_TOKEN), // Menggunakan const
284     password: None,
285     keep_alive_interval: Some(Duration::from_secs(30)),
286     ..Default::default()
287 };
288
289 let mqtt_connected = Arc::new(AtomicBool::new(false));
290
291 // --- MQTT Callback Handler (untuk menangani RPC OTA) ---
292 let mqtt_callback = {
293     let mqtt_connected = mqtt_connected.clone();
294
295     move |event: EspMqttEvent<'_>| {
296         use esp_idf_svc::mqtt::client::EventPayload;
297
298         match event.payload() {
299             EventPayload::Connected(_) => {
300                 log::info!(" MQTT connected");

```

```

301         mqtt_connected.store(true, Ordering::SeqCst);
302     }
303     EventPayload::Received { topic, data, .. } => {
304         let payload_str = std::str::from_utf8(data).
unwrap_or("");
305         log::info!("    Payload diterima. Topic: {:?},
Data: {}", topic, payload_str);
306
307         if let Some(topic_str) = topic {
308             // ⚡ Logic RPC Request untuk OTA
309             if topic_str.starts_with("v1/devices/me/rpc/
request/") {
310                 let parts: Vec<&str> = topic_str.split('
/').collect();
311                 if let Some(request_id) = parts.last() {
312                     log::info!(" ✓ Menerima RPC
request_id: {}", request_id);
313
314                     if let Ok(json) = serde_json::
from_str::<serde_json::Value>(payload_str) {
315                         let mut ota_url_owned = None;
316
317                         // Cek apakah ada parameter
ota_url di dalam RPC
318                         if let Some(url_str) = json.get
("params").and_then(|p| p.get("ota_url")).and_then(|u| u.as_str()
) {
319                             // ✓ FIX E0597: Kloning
string agar thread baru memiliki data (owned)
320                             ota_url_owned = Some(url_str
.to_owned());
321                         }
322
323                         if let Some(url) = ota_url_owned
{
324                             log::info!(" ⚡ Dapat OTA
URL dari RPC: {}", url);
325                             send_rpc_response(request_id
, "success");
326

```

```

327                                     // ✓ FIX 2: Mencegah Stack
Overflow dengan mengalokasikan stack 10KB
328                                     std::thread::Builder::new()
329                                         .stack_size(10 * 1024)
330                                         .spawn(move || {
331                                             // Pass reference (&
str) to the owned String
332                                             ota_process(&url);
333                                         })
334                                         .expect("Gagal membuat
thread OTA");
335
336                                     return;
337                                     } else {
338                                         log::warn!(" ⚠ Payload RPC
diterima, tetapi \"ota_url\" tidak ditemukan.");
339                                     }
340                                     } else {
341                                         log::error!(" ⚠ Gagal mem-parse
JSON payload RPC.");
342                                     }
343
344                                     send_rpc_response(request_id, "
failure");
345                                     }
346                                 }
347                             }
348                         }
349                         EventPayload::Disconnected => {
350                             log::warn!(" ⚠ MQTT Disconnected!");
351                             mqtt_connected.store(false, Ordering::SeqCst);
352                         }
353                         _ => {}
354                     }
355                 }
356             };
357
358             // --- Inisialisasi MQTT Client (Non-static Callback) ---
359             let client = loop {
360                 let res = unsafe {
361                     EspMqttClient::new_nonstatic_cb(

```

```

362         TB_MQTT_URL ,
363         &mqtt_config,
364         mqtt_callback.clone(),
365     )
366 };
367
368 match res {
369     Ok(c) => {
370         unsafe { MQTT_CLIENT = Some(c) };
371
372         if let Some(c_ref) = get_mqtt_client() {
373             while !mqtt_connected.load(Ordering::SeqCst) {
374                 log::info!(". . . Menunggu MQTT connect...");
375                 thread::sleep(Duration::from_millis(500));
376             }
377             log::info!("    MQTT Connected!");
378
379             // Subscribe ke topik RPC untuk menerima
perintah OTA
380             c_ref.subscribe("v1/devices/me/rpc/request/+",
QoS::AtLeastOnce).unwrap();
381
382             // LAPORAN INI KRUSIAL UNTUK OTA (melaporkan
versi dan status awal)
383             publish_fw_version();
384             publish_fw_state("IDLE");
385
386             break c_ref;
387         } else {
388             log::error!(" ⚠️ Gagal mendapatkan referensi
client setelah koneksi.");
389             thread::sleep(Duration::from_secs(5));
390             continue;
391         }
392     }
393     Err(e) => {
394         log::error!(" ⚠️ MQTT connect gagal: {:?}" , e);
395         thread::sleep(Duration::from_secs(5));
396     }
397 }
398 };

```

```

399
400 // --- Inisialisasi sensor DHT22 ---
401 let mut pin = PinDriver::input_output_od(peripherals.pins.gpio4)
?;
402 let mut delay = Ets;
403
404 // --- Loop utama kirim data (Interval 60 detik) ---
405 loop {
406     // Ambil waktu sekarang
407     let systime = EspSystemTime {}.now();
408     let secs = systime.as_secs() as i64;
409     let nanos = systime.subsec_nanos();
410
411     let naive = NaiveDateTime::from_timestamp_opt(secs, nanos as
u32).unwrap_or(NaiveDateTime::from_timestamp_opt(0, 0).unwrap())
;
412     // ✓ FIX E0308: Meminjam (&) nilai naive karena
from_utc_datetime membutuhkan referensi.
413     let utc_time = Utc.from_utc_datetime(&naive);
414     let wib_time = utc_time + ChronoDuration::hours(7);
415     // ✓ Fix Chrono API warning
416     let ts_millis = naive.and_utc().timestamp_millis();
417     let send_time_str = wib_time.format("%Y-%m-%d %H:%M:%S").
to_string();
418
419     // Baca sensor DHT22
420     match Reading::read(&mut delay, &mut pin) {
421         Ok(Reading {
422             temperature,
423             relative_humidity,
424         }) => {
425             // Siapkan payload JSON
426             let payload = json!({
427                 "send_time": send_time_str, // waktu kirim dari
ESP (WIB)
428                 "ts": ts_millis, // waktu epoch (
untuk analisis ThingsBoard)
429                 "temperature": temperature,
430                 "humidity": relative_humidity
431             });
432

```

```

433         let payload_str = payload.to_string();
434
435         // Kirim data Telemetry
436         match client.publish(
437             "v1/devices/me/telemetry",
438             QoS::AtLeastOnce, // Menggunakan QoS 1 untuk
telemetry
439             false,
440             payload_str.as_bytes(),
441         ) {
442             Ok(_) => log::info!("    Data terkirim (T: {} C ,
H: {}%): {}", temperature, relative_humidity, payload_str),
443             Err(e) => log::error!("    ✖  Gagal publish ke MQTT
: {:?})", e),
444         }
445     }
446     Err(e) => log::error!("    🚩  Gagal baca DHT22: {:?})", e),
447 }
448
449 // Delay diubah menjadi 60 detik
450 thread::sleep(Duration::from_secs(60));
451 }
452 }

```

Listing A.1: main.rs

Cargo.toml

```
1 [package]
2 name = "streamdht"
3 version = "0.1.0"
4 authors = ["zuhair"]
5 edition = "2021"
6 resolver = "2"
7 rust-version = "1.77"
8
9 [[bin]]
10 name = "streamdht"
11 harness = false # Do not use the built-in cargo test harness ->
    resolves rust-analyzer errors
12
13 [profile.release]
14 opt-level = "s"
15
16 [profile.dev]
17 debug = true      # Symbols are nice and they don't increase the
    size on Flash
18 opt-level = "z"
19
20 [features]
21 default = []
22 experimental = ["esp-idf-svc/experimental"]
23
24 [dependencies]
25 log = "0.4"
26 esp-idf-svc = "0.51"
27 rand = "0.8"
28 anyhow = "1.0"
29 heapless = "0.8"
30 serde_json = "1.0"
31 dht-sensor = "0.2"
32 chrono = { version = "0.4", features = ["clock"] }
33 embedded-svc = { version = "0.28.1" } # FIX: Disinkronkan dengan
    esp-idf-svc 0.51
34
35 # --- Optional Embassy Integration ---
36 # esp-idf-svc = { version = "0.51", features = ["critical-section",
```

```

    "embassy-time-driver", "embassy-sync"] }
37
38 # If you enable embassy-time-driver, you MUST also add one of:
39 # embassy-time = { version = "0.4.0", features = ["generic-queue-8"]
    }
40 # embassy-executor = { version = "0.7", features = ["executor-thread
    ", "arch-std"] }
41
42 # --- Temporary workaround for embassy-executor < 0.8 ---
43 # esp-idf-svc      = { version = "0.51", features = ["embassy-time-
    driver", "embassy-sync"] }
44 # critical-section = { version = "1.1", features = ["std"], default-
    features = false }
45
46 [build-dependencies]
47 embuild = "0.33"

```

Listing A.2: Cargo.toml

Build.rs

```

1 fn main() {
2     embuild::espidf::sysenv::output();
3 }

```

Listing A.3: Build.rs

BIBLIOGRAFI

- [1] Casquilho, M. (2022). A web-based superabsorbent polymer solver in simple science: PHP, Python, Fortran, and GNUPlot together. *Journal of Computational Science Education*, 13(2), 55–63.
- [2] Rocha, P., Santos, L., & Carvalho, R. (2023). Acceptance sampling in quality control: From theory to the web. *International Journal of Quality & Reliability Management*, 40(5), 1032–1048.
- [3] Frank, J., Miller, S., & Rossi, L. (2025). Ariel OS: An embedded Rust operating system for networked sensors and multi-core microcontrollers. *Embedded Systems Letters, IEEE*.
- [4] Annas, N., Rahman, A., & Kamarudin, S. (2024). Cloud-based IoT system for real-time harmful algal bloom monitoring: Seamless ThingsBoard integration via MQTT and REST API. *IEEE Access*, 12, 33021–33033.
- [5] Hidayat, M., Rini, D., & Rahmad, S. (2024). Design and implementation of temperature and humidity monitoring and control system in IoT-based chicken eggs incubator. *International Journal of Advanced Computer Science and Applications (IJACSA)*, 15(1), 120–128.
- [6] Song, L., Zhang, K., & Wu, J. (2023). Design of factory environmental monitoring system based on ESP32. *Sensors and Applications*, 23(4), 223–232.
- [7] Hakim, R., Pratama, Y., & Siregar, A. (2022). Design of IoT-based oyster mushroom monitoring and automation system prototype. *International Journal of Innovative Science and Research Technology*, 7(9), 182–189.
- [8] Bujal, N., Ahmad, F., & Rahim, H. (2024). Development of IoT-based compact mushroom cultivation monitoring system. *Indonesian Journal of Electrical Engineering and Computer Science*, 23(2), 425–432.
- [9] Widiyanto, A., Nugraha, D., & Fajar, P. (2022). Development of internet of things-based instrument monitoring application for smart farming. *Journal of Smart Agriculture Technology*, 4(1), 12–19.
- [10] Raghavendra Rao, P., & Kumar, V. (2023). High-performance file searching with Rust: Parallelized indexing for enhanced computational efficiency. *International Journal of Computer Applications*, 182(47), 8–15.

- [11] Bautista, J. A., & Cruz, M. L. (2023). Humidity and temperature control system for oyster mushroom. *International Journal of Electrical and Computer Engineering (IJECE)*, 13(2), 2100–2108.
- [12] Sălăgean, M., & Zinca, D. (2020). IoT applications based on MQTT protocol. *Procedia Computer Science*, 176, 1225–1234. <https://doi.org/10.1016/j.procs.2020.09.142>
- [13] Kumar, A., Singh, R., & Patel, S. (2021). IoT-based energy efficient agriculture field monitoring and smart irrigation system using NodeMCU. *International Journal of Intelligent Systems and Applications in Engineering*, 9(3), 112–118.
- [14] Saini, A., Gupta, R., & Mehta, D. (2022). Low-cost IoT mesh network for real-time indoor air quality monitoring. *International Journal of Smart Sensor Networks*, 6(1), 1–9.
- [15] Subramanian, K. M., & Rajan, R. (2021). Measuring temperature and CO₂ emissions from soil respiration using a real-time system. *Environmental Monitoring and Assessment*, 193(8), 495. <https://doi.org/10.1007/s10661-021-09242-0>
- [16] Saraswati, I., Nurhasanah, S., & Wulandari, D. (2022). Monitoring system temperature and humidity of oyster mushroom house based on the internet of things. *International Journal of Scientific Research in Engineering and Management (IJS-REM)*, 6(7), 22–29.
- [17] Nizam, M. S. M., Ismail, R., & Hamzah, A. (2024). Real-time energy monitoring in renewable EV charging stations: An ESP32-based system integrating Modbus, MQTT, and ESP_NOW protocols. *IEEE Transactions on Smart Grid Applications*, 15(3), 312–320.
- [18] Rádai, R., & Kovácsházy, T. (2025). Real-time GPIO performance of modern programming languages for embedded Linux application development. *IEEE Embedded Systems Letters*.
- [19] Çakıl, F., Aksoy, N., & Tekdemir, İ. G. (2024). Real-time smart power distribution in electric vehicle chargers utilizing Rust programming for enhanced efficiency. *IEEE Access*, 12, 61101–61115.
- [20] Li, Z., Narayanan, V., Chen, X., Zhang, J., & Burtsev, A. (2024). Rust for Linux: Understanding the security impact of Rust in the Linux kernel. *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, 123–135.